



## Full length article

# Any-quantile probabilistic forecasting of short-term electricity demand: Fusing uncertainties from diverse sources

Slawek Smyl<sup>a</sup>, Boris N. Oreshkin<sup>b</sup>, Paweł Pełka<sup>c</sup>, Grzegorz Dudek<sup>c</sup>,\*

<sup>a</sup> Walmart, USA

<sup>b</sup> Amazon, Canada

<sup>c</sup> Department of Electrical Engineering, Czestochowa University of Technology, 42-200 Czestochowa, Al. Armii Krajowej 17, Poland

## ARTICLE INFO

Dataset link: [www.entsoe.eu/data/power-stats](http://www.entsoe.eu/data/power-stats),  
<https://github.com/boreshkinai/any-quantile>

## Keywords:

Short-term load forecasting  
 Probabilistic forecasting  
 Quantile forecasting  
 Multi-source uncertainty fusion  
 Cross-learning  
 Deep learning

## ABSTRACT

Power systems operate under significant uncertainty arising from diverse and dynamic factors such as fluctuating renewable energy generation, evolving consumption patterns, and complex market dynamics. Accurately forecasting electricity demand necessitates advanced methodologies capable of capturing these multifaceted uncertainties. Our work develops any-quantile probabilistic forecasting framework, which enables the generation of forecasts for arbitrary quantile levels at inference time using a single trained model. This constitutes a substantial methodological advancement over traditional quantile regression techniques, which typically require training a separate model for each quantile or limiting predictions to a fixed set of predefined quantile levels. We show that integrating this framework into state-of-the-art neural architectures, specifically ESRNN and N-BEATS, yields superior distributional forecasting performance in the context of short-term electricity demand. Additionally, we develop the general Bayesian theory of cross-learning and link its latent objects with the elements of our architectures, providing a Fusion theory foundation for cross-learning from multiple power systems.

Empirical validation utilizing a comprehensive dataset of hourly electricity demand from 35 European countries showcases the efficacy of our approach, demonstrating superior predictive performance and enhanced quantile forecasting accuracy.

## 1. Introduction

Extraneous stochastic factors, which include weather fluctuations, load variations and equipment failures, make probabilistic forecasting arguably the only viable approach to forecasting in power systems [1]. Concretely, these factors introduce uncontrollable random variability in power system operations, which can only be accounted for, at any particular point in time, by keeping a model of many possible futures using the tools of probabilistic calculus. Probabilistic calculus induces probabilistic forecasts that provide likely distribution of electricity demand for a specific point in the future or a set of likely demand trajectories in the context of multi-horizon forecasting. Power system operators can use these probabilistic forecasts to enhance operational efficiency, mitigate risks and ensure reliable and secure power supply. This correlates with the following range of the applications of forecasting in power systems: (i) planning and optimization, (ii) security assessment and management and (iii) energy trading. In the following, we discuss the motivations behind the use of probabilistic forecasting in these application areas in more detail.

**Planning and optimization** benefit from probabilistic forecasting by incorporating probabilistic forecasts of load demand and generation availability to minimize operating costs, maximize revenue and/or ensure reliable power supply via stochastic optimization and scheduling. For example, operating reserve can be planned for uncertainties to maintain grid reliability during peak demand periods, unexpected events or sudden changes in weather conditions. Concretely, the integration of probabilistic forecast information can reduce the predicted regulation range by about 12%–31% [2], which reduces the necessary reserve margin costs and helps better utilize limited system balancing resources. Wang et al. [3] show that the probabilistic forecast information can be transformed into dynamic reserve requirements, which is beneficial in terms of reducing the cost and the number of power balance and reserve violations compared to the static reserve requirements. Finally, probabilistic renewable energy generation forecasts enable the design of ramping products that provide flexible and efficient resources to balance the grid during periods of changing

\* Corresponding author.

E-mail addresses: [slaweks@hotmail.co.uk](mailto:slaweks@hotmail.co.uk) (S. Smyl), [boris.oreshkin@gmail.com](mailto:boris.oreshkin@gmail.com) (B.N. Oreshkin), [pawel.pelka@pcz.pl](mailto:pawel.pelka@pcz.pl) (P. Pełka), [grzegorz.dudek@pcz.pl](mailto:grzegorz.dudek@pcz.pl) (G. Dudek).

<https://doi.org/10.1016/j.inffus.2025.103637>

Received 3 April 2025; Received in revised form 16 July 2025; Accepted 14 August 2025

Available online 23 September 2025

1566-2535/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

generation patterns, ensuring the seamless integration of renewable energy into the power system [4].

**Security assessment and management** in power systems focus on identifying high-risk extreme events and applying preventive measures accordingly [5]. For example, probabilistic forecasts of electricity demand can be used to identify high-likelihood future load demand increase events and operators can take preventive measures such as activating additional generation resources, adjusting generation schedules, or implementing load shedding plans to ensure grid stability. Other stability risks such as voltage fluctuations, frequency deviations and power imbalances, can also be better managed using probabilistic forecasts.

**Energy trading** optimizes generation and consumption patterns, manages risks and maximizes economic benefits by buying and selling electricity in various markets, such as wholesale electricity markets, balancing markets and ancillary services markets [6]. Probabilistic forecasting provides valuable insights to energy traders for optimizing their trading strategies. It quantifies the range of possible outcomes and associated uncertainties in key parameters, such as electricity prices, load demand and generation availability, which are essential for trading decisions. For example, if a probabilistic load forecast indicates a higher probability of a peak load scenario, traders can adjust their generation schedules or contract additional capacity to meet the expected demand.

In our current work, we focus on the short-term electricity demand forecasting, which has prominent applications in energy trading as well as short-term planning and scheduling optimization. The rest of the section provides the review of the most relevant literature in probabilistic forecasting and its applications in power systems, as well as summarizes core motivation and contributions of our work.

### 1.1. Related work

**Probabilistic forecasting in power systems** has profound impact on operational security, results in quantifiable financial outcomes and as such has profound impact on decision making in energy sector as a whole. The concept of probabilistic load forecasting is comprehensively examined in [7], focusing on the progress in general methodology and the impact in practical applications. Practical applications encompass such areas as probabilistic load flow, unit commitment and reliability planning. In contrast to point forecasting, probabilistic load forecasting characterizes load variations by providing predictions in the form of probability density functions (PDFs), quantiles, or prediction intervals (PIs). Practical examples of the use of forecast uncertainty in grid management are illustrated in [1]. These include the employment of probabilistic data for grid security calculations, the optimization of unit commitment, the mitigation of risks associated with extreme conditions and events. Furthermore, the review of probabilistic forecasting in smart grid applications can be found in [8], while applications of probabilistic forecasting in solar power systems are reviewed in [9].

**Probabilistic forecasting approaches**, as delineated by Hong et al. [7], can be broadly subdivided into three primary categories: (i) input scenario simulation, (ii) post-processing involving residual simulation or forecast combination and (iii) model-dependent interval construction and probabilistic forecasting models. In the first category, multiple scenarios for input variables, such as temperature, are employed to generate various point forecasts. These forecasts are then aggregated to create a probabilistic forecast. For example, [10] employs semi-parametric additive models to estimate the relationships between demand and various driver variables, including temperatures, calendar effects and some demographic and economic variables. Subsequently, the distributions of annual and weekly peak electricity demands are forecasted by utilizing a mixture of temperature simulations, anticipated future economic scenarios and residual bootstrapping. In another approach, Taylor and Buizza [11] use neural networks (NNs) for estimating probabilistic load

forecasts, with weather ensemble predictions for multiple scenarios being integrated as input variables. Additionally, [12] introduces generic scenario-based probabilistic load forecasting models using an ensemble of regression trees. In this case, quasi-periodic time-series data are represented as matrices and the singular value decomposition is employed to generate temperature scenarios. Finally, [13] explores and empirically compares three basic methods for generating temperature scenarios: the fixed-date, shifted-date and bootstrap.

The second approach to generating probabilistic forecasts involves post-processing existing point forecasts. This is typically accomplished by applying the probability density function of residuals on top of existing point forecasts. For example, a common approach consists in fitting a normal distribution to forecasting errors [14]. The key takeaway of the study is that the quality of the underlying model plays a crucial role in ensuring the success of post-processing probabilistic load forecast. The literature also includes more advanced methods of residual simulation, such as the application of GARCH [15]. Moreover, residual simulation can be successfully combined with predicted model uncertainties as shown in [16]. This requires a more sophisticated modeling approach employing a hybrid ensemble of deep belief networks, bagging, boosting approaches and a  $k$ -nearest neighbor classifier. Similarly, ensemble forecasting can be used as a standalone approach to generating probabilistic forecasts: a set of point forecasts produced by individual models can be transformed into a PI via a quantile regression as shown in [17].

The third probabilistic forecasting category encompasses forecasting models capable of generating probabilistic forecasts directly. Bayesian non-parametric techniques such as quantile regression based on Gaussian process are popular in this category. For example, in [18] Gaussian process predicts load quantiles in the next time step and the probability density function is obtained through kernel density estimation (KDE). Similar forecasting approaches, involving feature selection, quantile modeling and KDE are employed in [19–21]. The lasso method acts as feature selector, while neural network models quantiles via quantile regression in [19]. [20] utilizes a copula model for feature selection, while combining a 1D Convolutional Neural Network (CNN) and a Gated Recurrent Unit (GRU) to produce quantile forecasts. In [21], quantile forecasts produced by different quantile regression neural models are transformed into PDFs through KDE. These PDFs are subsequently combined using a multilayer Gaussian mixture distribution. Li et al. [22] extend [20] by additionally incorporating a time attention mechanism. This model additionally allows for quantitative modeling of two types of uncertainty: epistemic and aleatoric, which is achieved through the use of Monte Carlo dropout and an improved loss function based on the Kullback–Leibler divergence. Similar KDE-based approaches include [23–25]. He et al. [23] propose a kernel-based support vector quantile regression model using Copula theory, which can quantify the uncertainty between inputs, such as power load and real electricity price. The forecasting results under different quantiles are fed into the KDE function. KDE can also be used on top of basic learners as a part of the meta-learning approach [24]. Here, a quantile regression LSTM meta-learner combines point forecasts produced by tree-based models, including random forest, gradient boosting decision tree and light gradient boosting machine. The PDF is determined using KDE modified by Gaussian approximation of quantiles. Similarly, a quantile meta-model, in the form of quantile linear regression or quantile random forest, can be constructed by stacking on top of the array of base models as shown in [25].

**Parametric and non-parametric probabilistic forecasting.** In addition to the mechanism used to construct the forecast, probabilistic forecasting approaches can be categorized into parametric, semi-parametric and non-parametric [26]. All of the models discussed thus far are representatives of the non-parametric family. Further examples of non-parametric methods are provided below. Their common trait is the use of advanced machine learning models in combination with the variants of pinball loss for supervision. For example, [27] employs

a Long Short-Term Memory (LSTM) network, [28] uses feed-forward network and [29] combines GRU and temporal fusion transformer, while all of them are supervised with pinball loss. Additionally, [29] adds quantile constraints and PI penalty terms to prevent quantile crossover and aid in constructing tighter PIs. Smyl et al. [30] combine exponential smoothing with a recurrent neural network (RNN) based on a novel recurrent cell, specifically designed to efficiently model both short and long-term dependencies within time-series data. The resulting probabilistic forecasts are presented in the form of three quantiles, corresponding to the median and the bounds of PI. Alfieri and De Falco [31] decompose the load time-series through wavelet decomposition and predict its components using quantile regression forest. The final load forecasts are reconstructed by interpolating quantile predictions to prevent quantile crossing.

A parametric approach to generating probabilistic forecasts assumes a certain PDF for the forecast distribution and involves a forecasting model that produces parameters of this PDF. In its simplest form, PDF can be represented by a normal distribution with mean and variance parameters. Gaussian processes are a viable option for slightly more general modeling [32]. A more heavy-tail model such as Student-t distribution can be used on top of MLP, DeepAR and Transformer models, as done in GluonTS library [33]. Another approach to handling multi-modality and asymmetry of forecast distribution is based on Gaussian mixture model [34].

## 1.2. Motivation and contributions

It is crucial to acknowledge the growing array of factors contributing to significant uncertainty in the energy sector. These encompass the dynamic expansion of distributed renewable energy sources, the evolution of smart grids, the advancement of energy storage systems, the increasing prevalence of electric vehicles, the adoption of extended demand response programs and the liberalized markets with increasingly intricate pricing policies. These factors must contend with sudden weather changes and extreme weather events, further complicating the landscape. The fusion of information from these diverse and dynamic sources of uncertainty is inherently complex, necessitating advanced methodologies to effectively manage and mitigate the resultant unpredictability in energy systems.

The value of probabilistic forecasting in the context of tackling these challenges in power systems applications has been clearly demonstrated in the literature. At the same time, there is a clear gap in the probabilistic forecasting accuracy, which is primarily related to (i) the prevalent use of simple non-state-of-the-art architectural backbones (LSTM/GRU/CNN) that lag behind in terms of point forecasting accuracy and (ii) the prevalent use of simple quantile regression training and inference mechanisms that lag behind in terms of probabilistic forecasting accuracy.

In this paper, we address the existing gap by demonstrating that integrating state-of-the-art neural architectures, ESRNN (Exponential Smoothing Recurrent Neural Network) and N-BEATS (Neural Basis Expansion Analysis for Time Series), with advanced any-quantile distributional training, alongside cross-learning information fusion from diverse sources, significantly enhances both point and probabilistic forecasting accuracies.

We selected these two architectures for modification because, as their original authors (ESRNN was proposed by Smyl in [35], and N-BEATS was introduced by Oreshkin et al. in [36]), we possess deep insight into their design and potential. Both architectures have demonstrated strong performance in various challenging forecasting tasks – most notably, ESRNN won the prestigious M4 forecasting competition – and have proven effective in the context of electricity demand forecasting [30,37].

In a nutshell, our research contributions can be summarized as follows:

1. We introduce a novel any-quantile probabilistic forecasting framework that enables the generation of arbitrary quantiles at inference time using a single trained model. The approach is model-agnostic and can be applied to various machine learning architectures.
2. We develop a Bayesian theory of cross-learning and establish its correspondence with architectural components, laying a theoretical foundation for fusion-based learning across multiple power systems.
3. We propose architectural modifications to the ESRNN forecasting algorithm expanding it with the any-quantile capability.
4. We propose architectural modifications to the N-BEATS forecasting algorithm expanding it with the any-quantile capability.

In the experimental section, we present empirical results for the short-term electricity demand forecasting task, validating the proposed contributions. The results demonstrate that the any-quantile-enabled ESRNN and N-BEATS models consistently outperform a range of baseline methods, including statistical approaches and state-of-the-art machine learning models, primarily neural forecasting architectures.

The rest of the paper is organized as follows. Section 4 formulates the problem and conveys analysis results. Section 5 describes proposed neural architectures solving any-quantile forecasting task. Section 6 describes our empirical findings. Finally, Sections 7 and 8 discuss our findings and conclude the paper.

## 2. Short-term distributional forecasting problem

We define the forecasting problem with a forecast horizon of length  $H$  and a historical time-series  $[y_1, \dots, y_T] \in \mathbb{R}^T$ , where  $T$  is the time stamp of the last observation. Forecaster function  $f_\theta : \mathbb{R}^{W \times 1} \rightarrow \mathbb{R}^H$ , parameterized with  $\theta \in \Theta$ , takes the *lookback window*  $\mathbf{x} \in \mathbb{R}^W = [y_{T-W+1}, \dots, y_T]$  of length  $W \leq T$  ending with the last observed value  $y_T$ , along with quantile  $q$  and predicts the vector of  $q$ th quantiles of future values  $\hat{\mathbf{y}}_{T,q} \in \mathbb{R}^H = [y_{T+1,q}, \dots, y_{T+H,q}]$  based on past observations. The indices of this window belong to the index set  $\Delta_T^n = \{T - W + 1, \dots, T\}$ . In the following, the forecast of the  $q$ th quantiles,  $\hat{\mathbf{y}}_{T,q}$ , is denoted as  $\hat{\mathbf{y}}_q$ , dropping time index  $T$  for brevity, unless specifically stated otherwise.

The accuracy of distributional forecasting is assessed using Continuous Ranked Probability Score (CRPS) defined as:

$$\text{CRPS}(\hat{F}, y_o) = \int_{\mathbb{R}} \left( \hat{F}(y) - \mathbb{1}\{y \geq y_o\} \right)^2 dy, \quad (1)$$

where  $y_o$  is the observed value,  $\hat{F}$  denotes the empirical predictive cumulative distribution function (CDF) determined from the predicted set of quantiles and  $\mathbb{1}$  denotes the indicator function.

## 3. From classical forecasting to learnable information fusion strategies

In the previous section, we introduced a forecasting framework. We now examine how this framework relates to the problem of information fusion — specifically, the fusion of statistical evidence and uncertainty patterns across structurally related but distinct forecasting tasks. To motivate this perspective, we contrast classical forecasting approaches with modern learnable models through the lens of information aggregation and reuse.

Classical statistical models such as ARIMA or ETS generate forecasts by modeling parameters  $\psi_i$  for each individual time series  $i$  based solely on its historical observations  $\mathbf{x}_i$ , under a manually selected model structure  $\mathcal{M}$  (e.g. ARIMA, ETS), leading to the following Bayesian posterior model (we leave out the constant normalizing terms for brevity):

$$p(\psi_i | \mathbf{x}_i; \mathcal{M}) \propto p(\mathbf{x}_i | \psi_i; \mathcal{M}) p(\psi_i; \mathcal{M}). \quad (2)$$

Here, model parameters  $\psi_i$  are estimated independently for each time series, without leveraging information from related series. In

effect, each forecast is produced in isolation, and cross-series structures, e.g. shared temporal dynamics, seasonality profiles, or uncertainty regimes, are ignored. This results in a strictly local inference procedure:

$$p(y_i | \mathbf{x}_i; \mathcal{M}) = \int_{\psi_i} p(y_i | \psi_i; \mathcal{M}) p(\psi_i | \mathbf{x}_i; \mathcal{M}) d\psi_i. \quad (3)$$

In contrast, neural forecasting models trained across collections of time series can learn to fuse statistical signals across series by extracting common patterns and leveraging shared structure. This is particularly valuable in large-scale forecasting settings where many related series (e.g. from different regions, products, or market segments) exhibit similar but non-identical behaviors. From a Bayesian viewpoint, classical models impose fixed priors over time series structure, while data-driven models can learn task-conditioned priors or directly amortize the inference of model parameters.

To formalize this, we introduce a latent variable  $\phi$  that encodes shared statistical knowledge learned from a population of time series  $\mathcal{D}_{-i} = \{\mathbf{x}_j\}_{j \neq i}$ . This leads to the following hierarchical Bayesian formulation:

$$p(\psi_i, \phi | \mathcal{D}) \propto p(\mathbf{x}_i | \psi_i) p(\psi_i | \phi) p(\mathcal{D}_{-i} | \phi) p(\phi). \quad (4)$$

Here,  $p(\psi_i | \phi)$  defines a learned, task-conditional prior over per-series parameters. In practice, this can be instantiated via time-series-specific latent variables such as level and seasonality parameters (see Eqs. (11), (12) in Section 5.1). The term  $p(\mathcal{D}_{-i} | \phi)$  captures the global structure shared across the forecasting domain. This global structure can be instantiated by expressive neural backbones (e.g., RNNs or N-BEATS) co-trained across the collection of time series, as detailed in Sections 5.1 and 5.2.

This formulation captures the essence of cross-learning: forecasts for each series are informed not only by their local history but also by statistical structure extracted from related series, yielding the following global inference procedure:

$$p(y_i | \mathbf{x}_i, \mathcal{D}_{-i}) = \int_{\psi_i, \phi} p(y_i | \psi_i, \phi) p(\psi_i, \phi | \mathcal{D}) d\psi_i d\phi. \quad (5)$$

In this sense,  $\phi$  serves as an *information fusion variable*, encoding and propagating inductive bias across tasks. Jointly learning  $\phi$  and per-series parameters  $\psi_i$  enables information integration across multiple axes – time series identity, temporal resolution, and predictive uncertainty – yielding a flexible and expressive forecasting paradigm.

#### 4. Any-quantile forecasting framework

The Bayesian formulation above provides a principled foundation for learning to forecast by fusing information across related time series. In particular, it highlights how shared structure, captured via the latent variable  $\phi$ , can be integrated into a forecasting model to support transferability, generalization, and robustness. Building on this foundation, we now introduce a concrete instantiation of this paradigm through our *any-quantile forecasting framework*. This framework implements the fusion mechanism via a shared neural backbone (representing  $\phi$ ) and a flexible any-quantile approach shown in Fig. 1, enabling probabilistic forecasts at arbitrary quantile levels using a single model.

##### 4.1. Any-quantile learning

The proposed general methodology enabling any-quantile neural network forecasting is inspired by [38] and is depicted in Fig. 1. The key feature of any-quantile forecasting is that both the neural network and the loss function accept quantile level  $q$  as input. Suppose  $y$  is observed value,  $\hat{y}_q$  is the predicted  $q$ th quantile and the neural network is trained using quantile loss, defined as:

$$\rho(y, \hat{y}_q) = \begin{cases} (y - \hat{y}_q)q & \text{if } y \geq \hat{y}_q \\ (y - \hat{y}_q)(q - 1) & \text{if } y < \hat{y}_q. \end{cases} \quad (6)$$

Then the learning procedure defined based on this framework will possess interesting theoretical properties. In the following analysis we consider for simplicity and without loss of generality that  $\mathbf{x}$  is the observed time-series history and  $y$  is the unobserved scalar future value that we would like to forecast using a neural forecaster  $f_\theta$  trained on a  $N$ -sample dataset of  $(\mathbf{x}, y)$  tuples derived from the unknown joint probability density function  $p(\mathbf{x}, y)$ . Assuming that at training time quantile probability  $q$  is generated uniformly at random in  $[0, 1]$  in the mini-batch of size  $B$  and the following stochastic gradient descent (SGD) update, based on the quantile loss, is used to update neural forecaster parameters  $\theta$  (representing both per-series and information fusion parameters,  $\psi_i$  and  $\phi$ , respectively), at iteration  $k$ :

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta \frac{1}{B} \sum_i \rho(y_i, f_\theta(\mathbf{x}_i, q_i)), \quad (7)$$

then the sequence of  $\theta$  updates linearly converges to the solution of optimization problem on the full training dataset of size  $N$  [39]:

$$\theta^* = \arg \min_{\theta \in \Theta} \frac{1}{N} \sum_i \rho(y_i, f_\theta(\mathbf{x}_i, q_i)). \quad (8)$$

By the strong law of large numbers, as  $N$  increases without bound, the sum in the last equation converges to the following with probability 1:

$$\mathbb{E}_{\mathbf{x}, y} \mathbb{E}_q \rho(y, f_\theta(\mathbf{x}, q)) = \mathbb{E}_{\mathbf{x}, y} \int_0^1 \rho(y, f_\theta(\mathbf{x}, q)) dq. \quad (9)$$

Recalling that CRPS can be alternatively defined via the inverse CDF  $F^{-1}$  of  $y$  via quantile loss integral of the form [40]:

$$\text{CRPS}(\hat{F}, y) = 2 \int_0^1 \rho(y, F^{-1}(q)) dq, \quad (10)$$

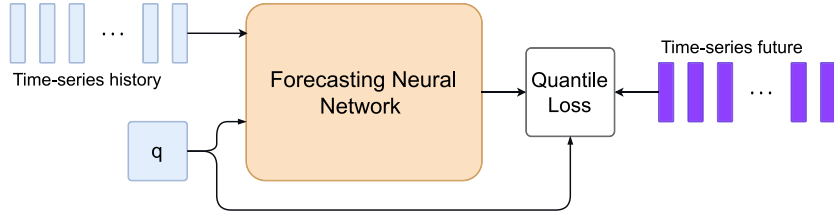
we establish a correspondence between the mini-batch update in Eq. (7), its expected optimization loss in Eq. (9), and the integral form of CRPS in Eq. (10), thereby linking the model output  $f_\theta(\mathbf{x}, q)$  to the inverse of the posterior cumulative distribution function  $P_{y|\mathbf{x}}$ . That is, for a uniformly sampled quantile  $q \sim \mathcal{U}(0, 1)$ , the prediction  $\hat{y}_q = f_\theta(\mathbf{x}, q)$  corresponds to a sample from the posterior distribution  $p(y|\mathbf{x})$ .

#### 5. Any-quantile forecasting models

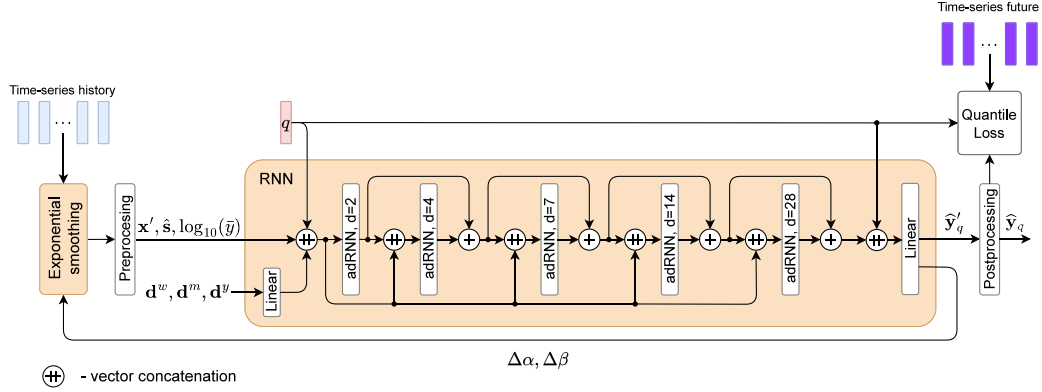
This section describes our main architectural contributions, showing the application of the general any-quantile forecasting methodology outlined in Section 4 to two popular neural forecasting architectures: ESRNN [41] and N-BEATS [36]. ESRNN and N-BEATS represent very different architectural approaches to the forecasting problem. The former is based on recurrent neural network design while the latter has multi-layer perceptron as its core. It is fair to say that these neural network architectures are orthogonal in their way of modeling time series. One of the objectives of our study is to present evidence that the proposed any-quantile learning framework generalizes well over different architectures. In the remainder of this section, we provide architectural design approaches for these two orthogonal neural architectures.

Both proposed models are trained in a cross-learning (or global learning) regime, where a single model is jointly trained on multiple related time series rather than on each series in isolation. By pooling information from diverse but related series, cross-learning enables the model to better generalize and improve forecasting performance across all included series [42,43]. This approach allows the model to capture common structures and dependencies, which is particularly valuable for forecasting electricity demand across multiple countries. In our case, by fusing time-series data from 35 national power systems, the models are exposed to a broad spectrum of uncertainties arising from regional weather patterns, consumption behaviors, and market dynamics. This fusion of diverse sources of uncertainty during training enables the models to learn more robust probabilistic representations, which is crucial for accurate and reliable quantile-based forecasting. In Section 6, we provide empirical evidence that both ESRNN and N-BEATS, when enhanced with the any-quantile learning mechanism and trained in this cross-learning setting, achieve state-of-the-art distributional accuracy in short-term electricity demand forecasting.





**Fig. 1.** Any-quantile forecaster training methodology. The network accepts time-series history and a quantile level. The quantile level is generated randomly for each training sample, according to a pre-specified distribution (e.g. uniform or Beta) and is used both as neural network input and as the supervision signal in the quantile loss.



**Fig. 2.** AQ-ESRNN architecture.

### 5.1. AQ-ESRNN

The any-quantile enabled ESRNN architecture, which we present in this section and call AQ-ESRNN is derived from ESRNN [41], a hybrid of Exponential Smoothing and a Recurrent Neural Network. Hence, preprocessing, postprocessing and the general architectural approach of AQ-ESRNN and ESRNN are very similar. Therefore, in our current exposition we provide only necessary details and refer interested reader to [41] for a more in-depth elucidation.

**Exponential smoothing and pre-processing.** In the context of the short-term electricity forecasting, we observe that the hourly time-series exhibit several seasonalities including daily (24 h), weekly (7 days, 168 h), yearly (changing seasons). The high-frequency seasonalities are explicitly treated in AQ-ESRNN by the exponential smoothing and pre-processing blocks shown in Fig. 2 using the  $w = 168$  point lag, while the yearly seasonality is dealt with by appending to the input an embedded week number. The exponential smoothing block is based on a version of Holinters multiplicative Exponential Smoothing (ES) model [44]:

$$\begin{aligned} l_{t,\tau} &= \alpha_t \frac{y_\tau}{s_{t,\tau}} + (1 - \alpha_t) l_{t,\tau-1}, \\ s_{t,\tau+w} &= \beta_t \frac{y_\tau}{l_{t,\tau}} + (1 - \beta_t) s_{t,\tau}, \end{aligned} \quad (11)$$

where  $y_\tau$  is the system load at hour  $\tau$ ,  $l_{t,\tau}$  is a level component,  $s_{t,\tau}$  is a weekly seasonal component and  $\alpha_t, \beta_t \in [0, 1]$  are smoothing coefficients. Note that in contrast to the standard ES, the smoothing coefficients  $\alpha_t, \beta_t$  change with time  $t$ . They are calculated from the RNN outputs,  $\Delta\alpha_t$  and  $\Delta\beta_t$ :

$$\begin{aligned} \alpha_{t+1} &= \sigma(\alpha_0 + \Delta\alpha_t), \\ \beta_{t+1} &= \sigma(\beta_0 + \Delta\beta_t). \end{aligned} \quad (12)$$

Here  $\alpha_0, \beta_0$  are initial values of the smoothing coefficients and  $\sigma$  is a sigmoid function, which maintains the coefficients within  $(0, 1)$  range.

Note in Fig. 2 that the RNN produces not only the forecasts of the target quantile,  $\hat{y}_q$ , but also corrections to the ES parameters,  $\Delta\alpha_t$  and  $\Delta\beta_t$ . These corrections are used to dynamically update the seasonal

component  $s_{t,\tau}$ , which is then applied during the preprocessing step – see (13). In this way, the model leverages the RNN to dynamically adjust the ES parameters, effectively optimizing the preprocessing step. This mechanism is designed to enhance the performance of the RNN in quantile forecasting and, consequently, to improve the overall forecasting accuracy.

**Recurrent NN backbone** is a deep residual RNN, which we call RNN for simplicity. RNN has several inputs, including the randomly generated quantile level  $q$  and a normalized, deseasonalized vector of the past  $w = 168$  values:

$$x_\tau = \log \frac{y_\tau}{\bar{y}_t \hat{s}_{t,\tau}}, \quad (13)$$

where  $\tau \in \Delta_t^{in}$ ,  $\Delta_t^{in}$  denotes the input rolling window, which is adjacent to the output window  $\Delta_t^{out}$  of length  $H = 48$  covering the forecast horizon,  $\bar{y}_t$  is the average value in  $\Delta_t^{in}$  and  $\hat{s}_{t,\tau}$  is the seasonal component produced by ES.

The input vector concatenates  $q$  and  $\mathbf{x}'_t = [x_\tau]_{\tau \in \Delta_t^{in}}$  with: (i) date features,  $\mathbf{d}_t^w \in \{0, 1\}^7$ ,  $\mathbf{d}_t^m \in \{0, 1\}^{31}$  and  $\mathbf{d}_t^y \in \{0, 1\}^{52}$ , i.e. binary one-hot vectors encoding day of the week, day of the month and week of the year, respectively; (ii) seasonality components,  $\hat{s}_t$ , i.e. a vector of 48 seasonal components predicted by ES for the output period  $t$  reduced by 1 and (iii) logarithm of the normalizer,  $\log_{10}(\bar{y}_t)$ .

The input is fed to the deep dilated RNN architecture consisting of 5 recurrent layers as shown in Fig. 2. RNN employs attentive dilated RNN cells, adRNNCells, introduced in [41]. They possess a receptive field whose size is determined by a dilation parameter ( $d \geq 1$ ). Moreover, they incorporate an attention mechanism that dynamically adjusts the weighting of input information, making them particularly effective at capturing complex dynamics and seasonalities. Larger dilations are assigned to top layers, i.e.  $d = 2, 4, 7, 14, 28$ . The dilations are in day units, as we step in time by 24 h. Residual connections flowing around each adRNNCell are added to avoid vanishing gradient problem. Finally, an important feature of the architecture is that the input containing a reference to the quantile probability  $q$  is repeatedly appended to the input of each layer, which improves the distributional forecasting accuracy according to our ablation studies in Section 6.7.

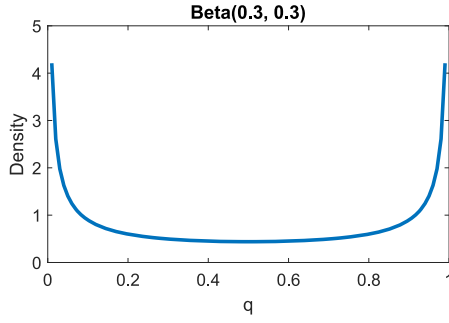


Fig. 3. Beta(0.3, 0.3) distribution used for the training of AQ-ESRNN. It oversamples the quickly changing, and important, edge areas.

Comparing the ESRNN and AQ-ESRNN architectures, we note that although the output size of AQ-ESRNN is significantly smaller (it forecasts a single quantile rather than multiple quantiles, e.g., five in the case of ESRNN), it is nonetheless deeper, employing five recurrent cells compared to three in ESRNN. This reflects the fact that forecasting any arbitrary quantile on demand is a more challenging task.

**Postprocessing.** The postprocessing component converts the RNN output, represented as a vector of  $H = 48$  normalized and deseasonalized  $q$ th quantiles,  $\hat{y}'_{q,t}$ , into actual quantiles,  $\hat{y}_{q,t}$ , using inverted Eq. (13):

$$\hat{y}_{q,t} = \exp(\hat{y}'_{q,t}) \bar{y}_t \hat{s}_{t,\tau}, \quad \tau \in \Delta_t^{out}. \quad (14)$$

We also found empirically that postprocessing the RNN forecast via quantile sorting noticeably improves distributional forecasting accuracy. For every horizon length (1 to 48) we sort the forecasted values for all quantiles and then match them with the sorted quantile probabilities. This prevents the quantile crossing and greatly improves the forecast accuracy (please refer to ablation studies for details).

**Training Methodology.** We use standard quantile loss function (6), operating on normalized values, i.e.  $y^n = y/\bar{y}$  and  $\hat{y}_q^n = \hat{y}_q/\bar{y}$ . A unique quantile probability  $q$  is sampled randomly for each time-series in the batch. It is worth mentioning that we found that AQ-ESRNN training benefits from non-uniform sampling of  $q$  with emphasis on the tails. We attribute this to the fact that ESRNN uses saturating activation functions, which lead to diminishing gradients at the tail of the distribution. Thus it is more challenging to fit well to very small (close to 0) and very large (close to 1) quantiles, because the quantile function is likely to change quickly in those regions. Additionally, these edge quantile levels are often very important for planning purposes. We conjecture that this can be compensated for by denser sampling of quantile probabilities in the tails. In particular, we use symmetric Beta distribution with both shape parameters less than 1, e.g., Beta(0.3, 0.3) (see Fig. 3).

## 5.2. AQ-NBEATS

Our exposition of the original N-BEATS architecture closely follows [37]. Each block of N-BEATS is a residual architecture consisting of the sequence of fully connected blocks with forecast/backcast forks at the end. The architecture runs a residual recursion over the entire input window and sums block outputs to make its final forecast. We assume that there are  $R$  residual blocks each having  $L$  hidden layers. If we refer to  $\mathbf{x} \in \mathbb{R}^W$  as the input of the architecture of length  $W$ , use residual block and layer superscripts ( $r$  and  $\ell$ , respectively) and denote the fully connected layer with weights  $\mathbf{W}^{r,\ell}$  and biases  $\mathbf{b}^{r,\ell}$  as  $\text{FC}_{r,\ell}(\mathbf{h}^{r,\ell-1}) \equiv \text{ReLU}(\mathbf{W}^{r,\ell} \mathbf{h}^{r,\ell-1} + \mathbf{b}^{r,\ell})$ , the operation of N-BEATS is described as follows:

$$\begin{aligned} \mathbf{x}^r &= \mathbf{x}^{r-1} - \hat{\mathbf{x}}^{r-1}, \\ \mathbf{h}^{r,1} &= \text{FC}_{r,1}(\mathbf{x}^r), \dots, \mathbf{h}^{r,L} = \text{FC}_{r,L}(\mathbf{h}^{r,L-1}), \\ \hat{\mathbf{x}}^r &= \mathbf{B}^r \mathbf{h}^{r,L}, \hat{\mathbf{y}}^r = \mathbf{F}^r \mathbf{h}^{r,L}. \end{aligned} \quad (15)$$

Here we have  $\hat{\mathbf{x}}^0 \equiv \mathbf{0}$ ,  $\mathbf{x}^0 \equiv \mathbf{x}$  and projection matrices have dimensions  $\mathbf{B}^r \in \mathbb{R}^{W \times d_h}$ ,  $\mathbf{F}^r \in \mathbb{R}^{H \times d_h}$ . Internal hidden dimension of each block is  $d_h$ . The final forecast is the sum of forecasts of all the residual blocks,  $\hat{\mathbf{y}} = \sum_r \hat{\mathbf{y}}^r$ . Given the structure of the network, there are three principal ways, depicted in Fig. 4 and described in detail below, that can be used to feed quantile  $q$  to N-BEATS.

**Input quantile concatenation scheme** is the easiest (does not require architectural modifications) and the most generic approach that is applicable to N-BEATS as well as any other forecasting architecture. In this scheme, called AQ-NBEATS-CAT and depicted in Fig. 4(a), quantile level is simply appended as one of the elements (e.g. the last element) in the input time-series history. The caveat here is that in the univariate forecasting scenario the original input has dimensions  $B \times W$  (batch size times the history length). In order to enable quantile computation for multiple quantiles  $Q$  for each time-series, the quantile-appended input will now have the form of 3D tensor of the size  $B \times Q \times (W + 1)$ , which multiplies the total computational cost of predicting  $Q$  quantiles at inference time by a factor of  $Q$ . Additionally, quantile level is restricted to the range  $[0, 1]$ . Hence, concatenating it with inputs of large magnitude can create dynamic range issues and make the network completely ignore the  $q$  input. Therefore, additional input normalization measures may have to be necessary to ensure effective quantile operation.

**Block quantile conditioning scheme** depicted in Fig. 4(b) is more specific to N-BEATS and implies architectural modification. In this scheme, which we call AQ-NBEATS-FiLM, we propose to use the FiLM conditioning method [45] to modulate the computation of each N-BEATS block with the representation derived from the quantile probability. In terms of inference path, the first and the last equation of AQ-NBEATS-FiLM block replicate original N-BEATS (15), whereas the middle part is modified as follows:

$$\begin{aligned} \mathbf{h}^{r,1} &= \text{FC}_{r,1}(\mathbf{x}^r), \\ \mathbf{c}_\alpha^r &= \text{LINEAR}_{r,\alpha}(\mathbf{q}), \quad \mathbf{c}_\gamma^r = \text{LINEAR}_{r,\gamma}(\mathbf{q}), \\ \mathbf{h}^{r,2} &= \mathbf{c}_\alpha^r + (1 + \mathbf{c}_\gamma^r) \mathbf{h}^{r,1}, \\ &\dots \\ \mathbf{h}^{r,L} &= \text{FC}_{r,L}(\mathbf{h}^{r,L-1}). \end{aligned} \quad (16)$$

FiLM induces both additive shift and multiplicative modulation of the internal time-series representation based on the target quantile probability. Since both multiplicative and additive components are included, this scheme is less sensitive to the issues that may arise from the lack of normalization, the problem we pointed out in relation with the AQ-NBEATS-CAT variant. Additionally, we hypothesize that the inclusion of quantile conditioning in each block should lead to better overall modeling of quantiles, since the architecture is effectively “reminded” of the quantile forecasting task in every block, whereas in AQ-NBEATS-CAT, the quantile hint is provided only once at the input and by the time the representation reaches the last block of a deep network it can effectively be washed away.

**Output quantile conditioning scheme** called AQ-NBEATS-OUT is depicted in Fig. 4(c). Here the bulk of the processing is done in a standard forecasting architecture, while quantile conditioning is done in the special output quantile conditioning block. This is a more computationally effective variant of AQ-NBEATS-FiLM. In AQ-NBEATS-FiLM, there is a computational cost for constantly “reminding” the architecture about the quantile it has to produce. Since quantile conditioning happens in every block, the architecture persistently works with  $B \times Q \times d_h$  tensors ( $d_h$  is the dimensionality of hidden layers), multiplying the total computational cost of predicting  $Q$  quantiles at inference time by a factor of  $Q$ . We hypothesize that the bulk of the work of representing a time-series can be done without quantile conditioning. Therefore, if we apply quantile conditioning only in the last block, as depicted in Fig. 4(c), we should be able to obtain comparable accuracy results while significantly saving on computation. For example, with a typical configuration of N-BEATS depth of 10–30 blocks, the computational

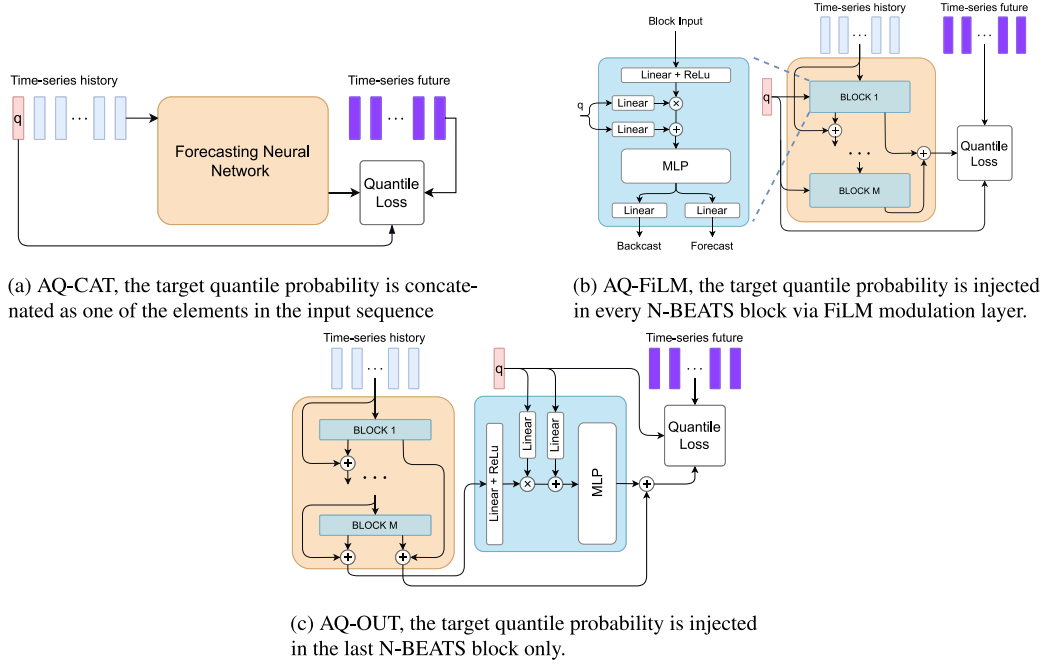


Fig. 4. Variants of implementing the any-quantile conditioning in N-BEATS.

cost of AQ-NBEATS-OUT will be 10–30 times smaller than that of AQ-NBEATS-FiLM.

**Training Methodology.** To train AQ-NBEATS, we use standard quantile loss function (6). A unique quantile probability  $q$  is sampled for each time-series in the batch, uniformly at random. The details of training methodology along with hyperparameter settings are provided in Section 6.4.

## 6. Empirical results

In this section, we conduct a comprehensive assessment of our proposed AQ-models in the context of short-term electricity demand forecasting across 35 European countries. The evaluation process unfolds as follows: we provide insights into the nature of the data, elucidate the methodologies involved in training and evaluation and describe models used as baselines. Subsequently, we present the results, delve into an ablation studies and conclude by engaging in a thorough discussion regarding the distinctive properties of our models.

### 6.1. Datasets

The real-world dataset utilized in this study is sourced from the ENTSO-E repository ([www.entsoe.eu/data/power-stats](http://www.entsoe.eu/data/power-stats)). This dataset encompasses hourly electricity demand data for 35 European countries and we have made this dataset publicly available along with the model code in our GitHub repository (To be released if the paper is accepted). The dataset contains a rich collection of time-series, each possessing distinct characteristics such as level, trend and variance varying over time, seasonalities (annual, weekly and daily) and random fluctuations of varying intensity. The inclusion of time-series with such diverse properties ensures a robust evaluation of the model's performance.

Fig. 5 provides a snapshot of the diverse energy consumption profiles across European countries. It illustrates the average demand and daily dispersion of hourly demands over the three-year period from 2016 to 2018. The average demand exhibits notable variability, ranging from 383 MWh for Montenegro (ME) to 58,941 MWh for Germany (DE). Similarly, the daily standard deviation of hourly demands varies from 56 MWh (Iceland, IS) to 7687 MWh (DE).

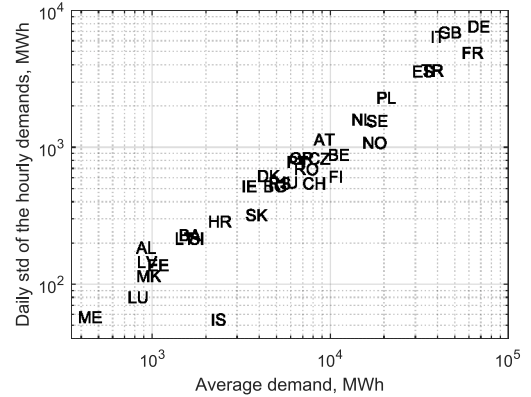


Fig. 5. Average electricity demand and daily dispersion demand in 35 European countries.

The variability in demand significantly influences the width of the probabilistic forecast distribution. We quantify this variability as the ratio of the standard deviation to average demand value:  $v = 100s/\bar{z}$ . We consider it in three contexts: as daily demand variations ( $v_d$ ), where  $\bar{z}$  and  $s$  denote the daily mean and standard deviation, respectively; as weekly variations ( $v_w$ ), where  $\bar{z}$  and  $s$  represent the weekly mean and standard deviation of daily means; and as yearly variations ( $v_y$ ), where  $\bar{z}$  and  $s$  express the yearly mean and standard deviation of weekly means. Fig. 6 shows the variation coefficients for 35 European countries. Notably, Iceland exhibits the least demand variation. The most pronounced daily variations ( $v_d > 17\%$ ) are observed in Albania, Italy, Latvia and Great Britain. For yearly variations, Norway, France, Sweden and Macedonia demonstrate the highest variability ( $v_y > 18\%$ ). Weekly variations are generally milder than daily and yearly ones, with  $v_w < 13\%$ . Italy, Germany, Austria and Poland show the strongest weekly variations. It is crucial to highlight that the pattern of electricity demand variation evolves over time, presenting additional challenge for forecasting models.

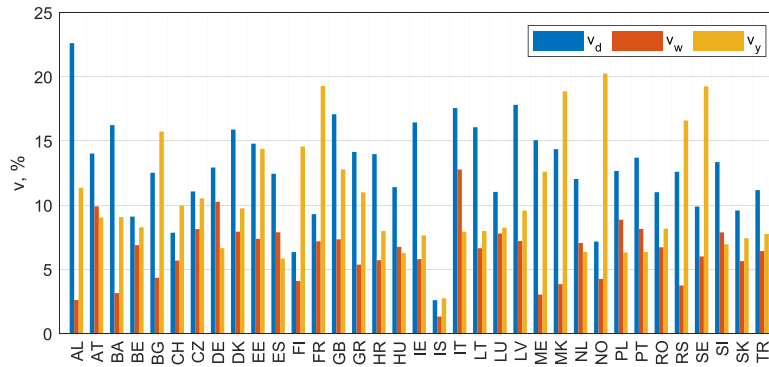


Fig. 6. Diversity of electricity demand in 35 European countries.

## 6.2. Training and evaluation setup

The dataset is split into train, validation and test subsets with the following boundaries: train dates 2006-01-01 to 2016-12-31, validation dates 2017-01-01 to 2017-12-31, test dates 2018-01-01 to 2018-12-31. In the training set, NaN (missing) values are forward-filled by replacing each missing value with the value from 168 h earlier (i.e., one full week) to enable their use during training. This is arguably the most suitable method for handling missing values when the time series exhibits strong high-frequency seasonality (weekly, with a 168-h cycle), along with an additional low-frequency seasonality (yearly). In the test set, NaN values (which occur during the final month of 2018 for Estonia and Italy, and the last two months of 2018 for Latvia) are excluded from metric calculations to avoid any information leakage.

We use the train and validation subsets to tune hyperparameters. Once the hyperparameters are determined, the model with best hyperparameter settings, re-trained on the train and validation sets, is used to report results on the test set. The prediction horizon is 48, or two days. A prediction horizon of this length is imposed by the requirements of power system operation. The system operator typically prepares forecasts for the next day well in advance, for example 12 h in advance according to [46].

For the final evaluation, we employ the ensemble of eight models, each sharing the same structure but initialized randomly and exposed to distinct, randomized sequences of input data (for a fair comparison, all baseline neural models are likewise evaluated using ensembles of eight models). The final forecasts are generated by taking the medians of all ensemble forecasts.

ESRNN model training and inference is performed on a PC with Intel i9-10980XE CPU and 256 GB of RAM. We train the ensemble (8 models) concurrently. In this setup the training of full models with 5 layers takes around 2–3 days. N-BEATS model training and inference are performed on the Dell Poweredge C4130 with 4xNVIDIA P100 GPU and 2 x Intel Xeon E5-2660 v3 @ 2.6 GHz processors. The training time in this hardware setup is 8 h and inference time is 10 min for 8-model N-BEATS AQ-OUT ensemble.

Statistical and machine learning baseline models are trained on eight-core CPU (AMD Ryzen 7 1700, 3.0 GHz, 32 GB RAM) resulting in the following compute times per model: ARIMA: 180 h; ES: 50 h; Theta: 30 h; QR: 58 h; NGBoost: 20 h; MLP: 6 min; DeepAR: 2 h and 40 min; Transformer: 3 h and 10 min; TFT: 2 h and 50 min; WaveNet: 7 h.

## 6.3. AQ-ESRNN training and hyperparameter settings

All AQ-ESRNN hyperparameters are adjusted by minimizing CRPS on the validation set. The objective function used to train the network is quantile loss (6). The model is implemented in PyTorch [47] and is trained for 6 epochs corresponding to around 1500 batch updates using Adam optimizer [48]. We use a schedule of increasing batch sizes.

Table 1

Settings of AQ-ESRNN hyperparameters and the hyperparameter search grid.

| Hyperparameter | Value                            | Grid                               |
|----------------|----------------------------------|------------------------------------|
| Batch size     | 2, changed to 5 in ep. 4         | changed to 5 in ep. 3, 4           |
| Epochs         | 6                                | 4, ..., 9                          |
| Learning rates | 0.003, /3 in ep. 5, /10 in ep. 6 | also /30 in ep. 7                  |
| LR multiplier  | 30                               | [10, 30]                           |
| BetaShape      | 0.3                              | [0.1, 0.2, 0.3, 0.5, 0.65, 0.8, 1] |
| Dilations      | [[2],[4],[7],[14],[28]]          | See Section 6.3                    |
| training steps | 50                               | [20, 30, 50]                       |
| c-state size   | 150                              | [100, 150, 200]                    |
| h-state size   | 70                               | [50, 70, 100]                      |
| $\alpha_0$     | 3.5                              | –                                  |
| $\beta_0$      | 0.3                              | –                                  |

Initially, we use batch size of 2 corresponding to 1750 updates per epoch and as batch size is increased to 5, the number of batch updates decreases to approximately 1450 per epoch. In addition to batch size scheduling, we use learning rate scheduling. On top of this, model contains global learnable parameters and time-series specific learnable parameters. For the latter, there is LR multiplier boosting the learning rate for per-series parameters. When experimenting with models having fewer layers, we observed earlier signs of overfitting. Therefore, for 3 and 4-layer models, we report results from epoch 5, while for 5-layer models, we report results from epoch 6.

Table 1 summarizes optimal hyperparameter settings of the AQ-ESRNN model, alongside the ranges explored during hyperparameter tuning process. A few additional remarks are in order to explain the fields of the table. *BetaShape* is a parameter utilized for a symmetrical Beta distribution, which is employed in generating the quantiles during training. *Dilations* describe the architecture of AQ-ESRNN. Each internal square bracket lists the dilations of layers in a particular block. For instance, [[2], [4], [7], [14], [28]] describes a system composed of 5 blocks, each with a single layer and a progressively increasing delay of 2, 4, 7, 14 and 28 time steps. In experiments with 3-layer architecture, dilations are set as [[2], [4], [7]]. For 4-layer architecture, we experimented with dilations like [[2], [4], [7], [14]], [[2], [4], [7], [7]] and [[2], [4], [7], [4]]. Similarly, for 5-layer architectures, we tested dilations such as [[2], [4], [7], [14], [28]], [[2], [4], [7], [14], [14]], [[2], [4], [7], [14], [7]] and [[2], [4], [7], [10], [14]]. *Training steps* – when a batch is formed from randomly chosen series, the RNN time steps initiate from a randomly selected step and persist for this number of steps. *C-state size* and *h-state size* – the cells employed are derived from LSTMs, but differ in several aspects, including c-state and h-state having different sizes. The c-state size must be larger than h-state size, because the cell output size equals c-state size minus h-state size.

## 6.4. AQ-NBEATS training and hyperparameter settings

AQ-NBEATS model is evaluated using the hyperparameter settings presented in Table 2. Similarly to AQ-ESRNN, the hyperparameters



**Table 2**  
Settings of AQ-NBEATS hyperparameters and the hyperparameter search grid.

| Hyperparameter                 | Value          | Grid             |
|--------------------------------|----------------|------------------|
| Epochs                         | 15             | [10, 15, 20]     |
| Loss                           | Quantile, (17) | [(17), (6)]      |
| Input Normalization            | No             | [Yes, No]        |
| Width ( $d_h$ )                | 1024           | [256, 512, 1024] |
| Blocks ( $R$ )                 | 30             | [5, 10, 20, 30]  |
| Layers ( $L$ )                 | 3              | [2, 3, 4]        |
| Sharing                        | False          | [True, False]    |
| Lookback period ( $w$ , hours) | 168            | [24, 168, 336]   |
| Batch size                     | 1024           | [256, 512, 1024] |
| Optimizer                      | Adam           | Adam             |
| Learning rate                  | 0.0005         | [0.001, 0.0005]  |

are adjusted by minimizing CRPS on the validation set. The objective function used to train the network is normalized quantile loss:

$$\rho_N(y, \hat{y}_q) = \begin{cases} (y - \hat{y}_q)q/y, & \text{if } y \geq \hat{y}_q, \\ (y - \hat{y}_q)(q - 1)/y, & \text{if } y < \hat{y}_q. \end{cases} \quad (17)$$

The model is implemented in PyTorch [47] using pytorch lightning framework [49] trained for 15 epochs using the Adam optimizer [48] with default settings using inverse square root learning rate annealing with maximum learning rate 0.0005 and warm-up period of 400 batches. Batch has size 1024 and the model takes the history of 168 points (24 h times 7 days) and predicts 48 points (two times 24 h) ahead in one shot.

### 6.5. Baseline models

We compare our AQ-models against a number of probabilistic forecasting baselines, encompassing both statistical and machine learning models, primarily neural network-based approaches. First, we list the baselines and in the ensuing paragraphs we explain important implementation details.

- **Naive**, a seasonal naive method: the forecasted demand profile for day  $t$  is the same as the profile for day  $t - 7$ .
- **ARIMA**, autoregressive integrated moving average model [50].
- **ES**, exponential smoothing model [50].
- **Theta**, dynamic optimized Theta model, DOTM v3 from [51].
- **QR**, quantile linear regression model [52].
- **NGBoost**, natural gradient boosting model [53].
- **MLP**, single-hidden-layer perceptron with sigmoid nonlinearities [54].
- **DeepAR**, autoregressive RNN model for probabilistic forecasting [55].
- **Transformer**, vanilla transformer [56].
- **WaveNet**, deep autoregressive model combining causal filters with dilated convolutions [57].
- **TFT**, temporal fusion transformer [58].

Naive, ARIMA, ES and Theta models generate point forecasts. To deal with triple seasonality in electricity demand time-series, we decompose these series into 24 distinct series, each representing a particular hour of the day. For each of these 24 series, a separate statistical model is utilized to forecast electricity demand, with a horizon of two. Consequently, all models produce a total of 48 forecasts for the subsequent two days. For Naive, ARIMA and ES, we assume that the point forecast represents mean of normal distribution and the standard deviation is estimated using the standard deviation of residuals [59]. To obtain quantiles, the models are first fitted using their respective R functions in the forecast package: `snaive()`, `auto.arima()` and `ets()`. Quantiles are then derived by setting the `level` parameter in the `forecast()` function to specific value defining PI. In contrast,

**Table 3**

Key performance metrics. Lower score is better. Best result is shown in bold.

|             | CRPS          | N-CRPS      | MAPE        |
|-------------|---------------|-------------|-------------|
| Naive       | 502.62        | 3.95        | 5.08        |
| ARIMA       | 353.49        | 2.83        | 3.74        |
| ES          | 325.80        | 2.64        | 3.48        |
| Theta       | 329.99        | 2.69        | 3.56        |
| QR          | 357.38        | 2.90        | 3.92        |
| NGBoost     | 329.21        | 2.72        | 3.75        |
| MLP         | 415.67        | 3.28        | 4.49        |
| DeepAR      | 378.46        | 2.97        | 3.95        |
| WaveNet     | 293.38        | 2.52        | 3.39        |
| Transformer | 380.41        | 2.83        | 3.72        |
| TFT         | 325.80        | 2.92        | 3.96        |
| AQ-ESRNN    | <b>195.94</b> | <b>1.72</b> | <b>2.32</b> |
| AQ-NBEATS   | 211.22        | 1.84        | 2.47        |

the Theta method estimates quantiles through bootstrapping, which involves simulating a sample of possible values of forecasts from the estimated model [60]. This technique is implemented in the `dotm()` function in the `forecTheta` package.

The QR implementation utilizes QuantileRegressor from the `sklearn.linear_model` Python library [61], while NGBoost forecasts are generated using NGBRegressor from the `ngboost` package. Both these models incorporate comprehensive feature sets consisting of calendar variables (hour of day, day of week, quarter, month, year, day of year, day of month, and week of year) and historical demand patterns. The historical features capture lagged demand values from  $t - 2$  to  $t - 21$ , representing 20 consecutive days of demand history preceding the forecast period. For each quantile level, a separate QR model is trained using the pinball loss function. In contrast, NGBoost generates quantile forecasts by estimating the parameters of a parametric probability distribution, specifically, the normal distribution in our case. Once the distribution parameters are predicted, the quantiles are obtained analytically from the corresponding inverse cumulative distribution function. NGBoost is trained by minimizing the negative log-likelihood of the target variable under the predicted distribution, using natural gradient descent.

Neural models employed in this study are sourced from GluonTS, a Python package designed for probabilistic time-series modeling, with a primary focus on deep learning models [33]. Specifically, the MLP, DeepAR and Transformer models are utilized to generate probabilistic forecasts through a parametric approach. These models have a projection layer that produces distribution parameters. Model fitting process involves the negative log-likelihood of the chosen distribution as the loss function. We use Student's t-distribution, which is the default setting. Once the model is fitted, quantiles can be derived from the learned parametric distribution. In contrast, WaveNet models the auto-regressive forecast probability distribution by outputting a softmax-induced categorical distribution over a fixed grid of quantiles. Finally, TFT simultaneously predicts multiple quantiles at each time step alongside a point forecast. Quantile forecasts are derived by applying a linear transformation on top of the TFT decoder embedding. During training, TFT learns by jointly minimizing the quantile loss aggregated across all quantile outputs. Arbitrary quantiles are derived from the fixed quantile grid via interpolation.

The hyperparameter settings of the baseline models are summarized in Tables A.6–A.9 in the Appendix.

### 6.6. Key quantitative results

Our key results are presented in Table 3. They include CRPS (1), normalized CRPS (N-CRPS) and MAPE for assessing point forecasts derived from distributional forecasts. Normalized version of CRPS is defined as follows:

$$\text{N-CRPS}(f_\theta, \mathbf{y}, \mathbf{x}) = 100 \frac{1}{C} \sum_c \frac{1}{HN_c Q} \sum_{i,h,q} \rho(y_{i,h,c}, f_\theta(\mathbf{x}_{i,c}, q)) / \bar{y}_c. \quad (18)$$

|             |    |    |    |    |    |    |    |    |    |    |    |   |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|---|----|
| Naive       | 0  | 0  | 1  | 1  | 0  | 0  | 0  | 0  | 1  | 0  | 2  | 0 | 0  |
| ARIMA       | 35 | 0  | 1  | 4  | 13 | 6  | 27 | 9  | 2  | 7  | 7  | 0 | 0  |
| ES          | 34 | 30 | 0  | 19 | 26 | 10 | 32 | 23 | 4  | 15 | 11 | 0 | 0  |
| Theta       | 34 | 27 | 10 | 0  | 27 | 6  | 33 | 23 | 4  | 13 | 7  | 0 | 0  |
| QR          | 34 | 8  | 3  | 4  | 0  | 6  | 27 | 11 | 3  | 6  | 7  | 0 | 0  |
| NGBoost     | 35 | 24 | 21 | 23 | 28 | 0  | 32 | 26 | 13 | 25 | 14 | 0 | 1  |
| MLP         | 35 | 2  | 1  | 2  | 3  | 0  | 0  | 4  | 1  | 2  | 5  | 0 | 0  |
| DeepAR      | 35 | 13 | 4  | 2  | 16 | 4  | 31 | 0  | 2  | 5  | 6  | 0 | 0  |
| WaveNet     | 34 | 31 | 28 | 30 | 30 | 13 | 33 | 33 | 0  | 30 | 20 | 0 | 0  |
| Transformer | 34 | 20 | 7  | 10 | 23 | 8  | 32 | 24 | 2  | 0  | 8  | 0 | 0  |
| TFT         | 33 | 25 | 21 | 22 | 26 | 13 | 29 | 27 | 13 | 23 | 0  | 0 | 0  |
| AQ-ESRNN    | 35 | 35 | 35 | 35 | 35 | 35 | 35 | 35 | 34 | 35 | 35 | 0 | 24 |
| AQ-NBEATS   | 35 | 35 | 35 | 35 | 35 | 34 | 35 | 35 | 35 | 35 | 35 | 5 | 0  |

Fig. 7. Results of the Diebolt-Mariano test based on the N-CRPS metric ( $\alpha = 0.05$ ). Individual cells count the number of countries the model represented on the y-axis is statistically more accurate than the model represented on the x-axis.

Here  $C = 35$  is the number of countries,  $H = 48$  is the number of horizons,  $Q = 100$  is the number of random test quantiles and  $N_c$  is the number of test samples per country. The second sum runs over samples in the country time series, horizons and quantiles.  $\bar{y}_c$  is the mean of a given country time-series. Normalized CRPS unifies results for different countries by normalizing over mean demand level  $\bar{y}_c$ . Therefore, all countries can be compared with each other and their impact in the average N-CRPS value shown in Table 3 is similar.

We see that the any-quantile forecasting task is challenging for classical time-series models as well as for the out-of-the-box machine learning models such as NGBoost and NNs from GluonTS. Among statistical models, ES emerges as top performer, while WaveNet leads the neural baselines. WaveNet slightly outperforms ES. Our novel any-quantile architectures, AQ-ESRNN and AQ-NBEATS, set new benchmarks in distributional forecasting, achieving the lowest CRPS. Additionally, when comparing mean absolute percentage errors (MAPE) as the performance measure for point forecasting, where the 0.5-quantile (median) is treated as a point forecast, the AQ-models surpass all baselines. Notably, AQ-ESRNN slightly outperforms AQ-NBEATS across both CRPS and MAPE metrics.

To assess the statistical significance of differences between the probabilistic forecasts generated by each pair of models, we conduct a Diebold–Mariano test [62] based on the N-CRPS metric computed for individual countries. The results, evaluated at a 0.05 significance level, are shown in Fig. 7, which counts the number of countries the model represented on the y-axis is statistically more accurate than the model represented on the x-axis. It is noteworthy that both our AQ-models demonstrate superior accuracy over baseline models for nearly all countries. Furthermore, AQ-ESRNN exhibits higher accuracy than AQ-NBEATS in 24 countries, while AQ-NBEATS outperforms AQ-ESRNN in 5 countries.

Fig. 8 presents the evaluation of PIs of varying sizes generated by the forecasting models. Bars indicate the proportion of observed values that fall within, below, or above the boundaries of the PIs predicted by the models, while the red dashed lines signify the theoretical target (for instance, for a 50%-PI, the targets are 50% within, 25% below and 25% above the PIs). In the context of In-PI charts, bars surpassing the target level signify PIs that are excessively broad, implying an overestimation of the uncertainty range. Conversely, bars falling short of target suggest PIs that are unduly narrow, indicating an underestimation of uncertainty. The pattern of too narrow PIs is particularly evident in the cases of the DeepAR and Transformer models, whereas

an inclination towards too broad PIs is primarily observed for 50%-PI in the statistical models, Naive, WaveNet and TFT. When examining the Below-PI and Above-PI charts, the most striking observation is that for DeepAR and Transformer the upper PI bounds are notably too low, with the bars in the Above-PI charts significantly above the target values. Conversely, WaveNet exhibits an opposite trend, with excessively high upper PI bounds. For broader PIs, specifically those at 98% and 99.8%, predicted PIs are often too tight, indicating challenges in forecasting extreme quantiles. In these scenarios, the TFT model exhibits superior performance. Excluding these extreme quantiles, the performance of our any-quantile models stands out, achieving PIs that closely approximate the targets.

To provide for a more comprehensive assessment of probabilistic forecasts, we additionally examine relative frequency (RF), also referred to as calibration or coverage [63].  $RF(q)$  quantifies the proportion of observed values  $y_i$  falling below or equal to the predicted  $q$ -quantile  $\hat{y}_{q,i}$  and is defined as follows:

$$RF(q) = \frac{1}{M} \sum_{i=1}^M \mathbb{1}\{y_i \leq \hat{y}_{q,i}\}. \quad (19)$$

For probabilistic forecasts to be precise and informative, they should exhibit an RF value close to the nominal probability level, with a reasonably small deviation from the realized values [64]. In essence, probabilistic forecasts are expected to capture uncertainty without significantly deviating from the actual series values. Ideally, predicted  $q$ -quantiles should surpass the realized values in  $100 \cdot q\%$  of cases, ensuring an RF of  $q$ . To evaluate the average deviation of  $RF(q)$  from the desired  $q$  across all assumed quantile probabilities  $q \in \Pi = \{0.001, 0.01, 0.02, \dots, 0.99, 0.999\}$ , we introduce the mean absolute RF error:

$$MARFE = \frac{1}{|\Pi|} \sum_{q \in \Pi} |RF(q) - q|. \quad (20)$$

Fig. 9 illustrates the RF charts, where the desired RF values are depicted with dashed lines. Each dot represents RF calculated for a specific country and probability  $q \in \Pi$ , while the red line signifies the mean RF value across all countries. From this figure, it is apparent that DeepAR, WaveNet, Transformer and TFT exhibit the most distorted RF distributions, displaying deviations from the desired probabilities in various directions. MARFE, corresponding to the deviation of the red line from the dashed line, exceeds 0.04 for these models. In contrast, Naive and statistical models demonstrate less scattered results, with a slightly s-shaped curve. The lowest MARFE is observed for QR (0.0104), followed by AQ-ESRNN (0.0157), MLP (0.0168), NGBoost (0.0174), Theta (0.0187) and AQ-NBEATS (0.0211). Notably, our any-quantile models accurately express the target probabilities, with RF curves closely aligned with the desired values and the average line almost coinciding with the dashed line.

Fig. 10 concludes our empirical study by providing the examples of probabilistic forecasts generated by our proposed AQ-models for Poland, Great Britain, France and Germany.

### 6.7. AQ-ESRNN ablation studies

Ablation studies of AQ-ESRNN are reported in Table 4, with the full model showcased in the first row. Initially, we explore the impact of reducing the number of layers (cells). It is evident that a larger number of layers enhances accuracy, albeit at the expense of increased computation time. Next, AQ-ESRNN-noQuantInMidLayers removes the quantile from the middle layers, while AQ-ESRNN-noInputInMidLayers eliminates all inputs from the middle layers. These architectural variants are depicted in Fig. 11a and b, respectively. Both of these simplified variants yield inferior results, demonstrating the value of “reminding” the target probability to the neural network at each block. Furthermore, AQ-ESRNN-dRNN variant represents the full model (5 layers) but employs simpler dRNN cells without the attention mechanism,

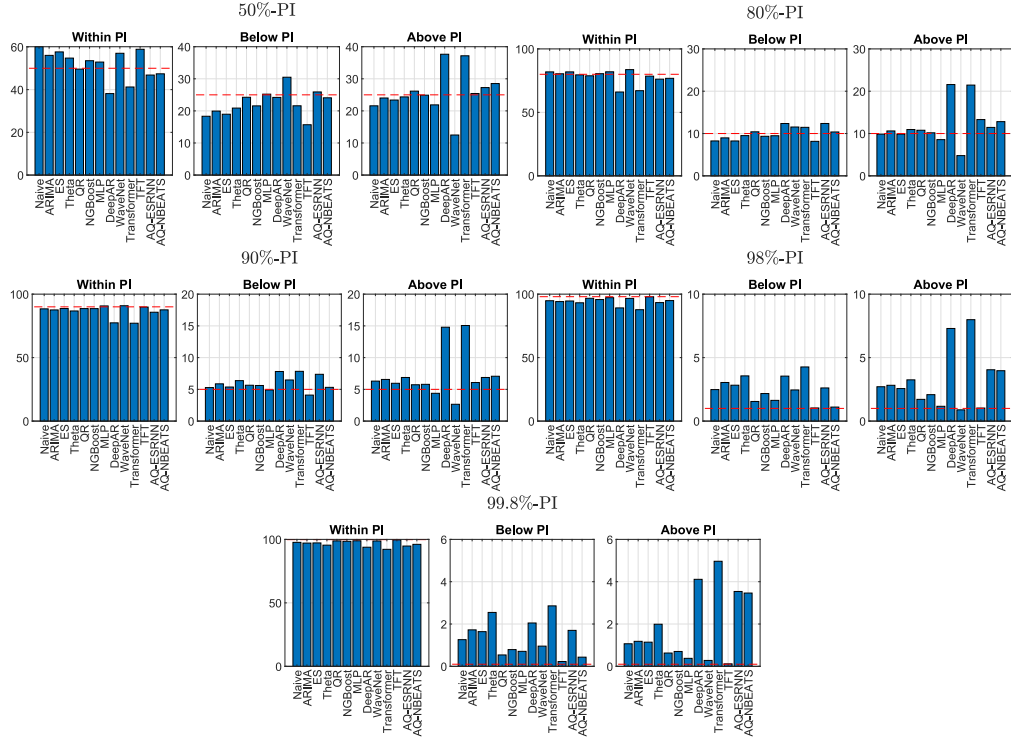


Fig. 8. Percentage of forecasts in, below and above the PI of different size. Red dashed line indicates the target value.

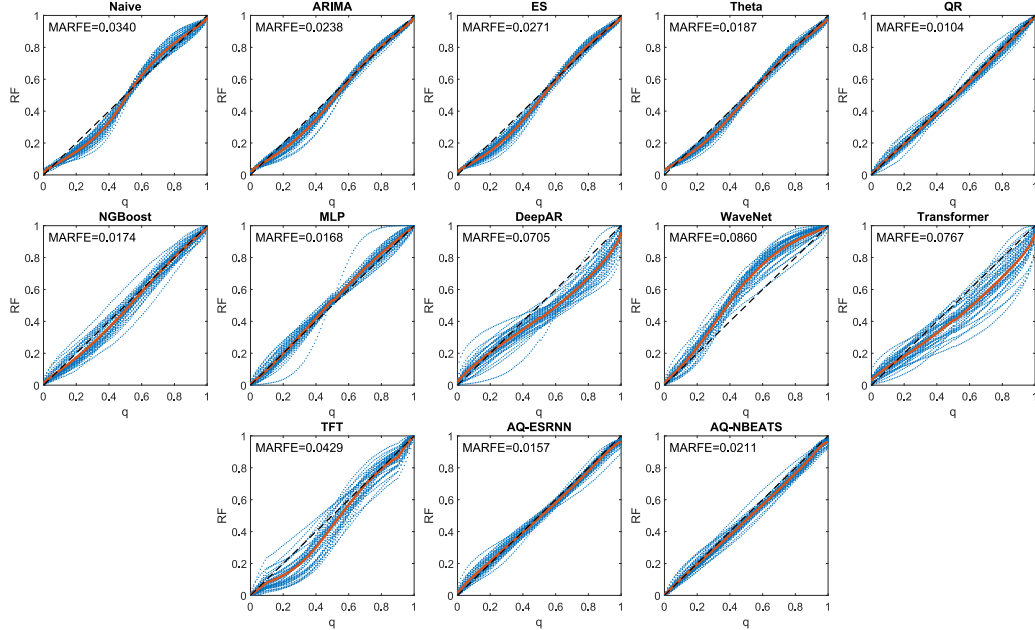


Fig. 9. Relative frequency.

as described in [41]. AQ-ESRNN-LSTM uses the standard LSTM cell. Simplifying the cell type leads to a deterioration in results. Lastly, AQ-ESRNN-unsorted is the full AQ-ESRNN model that does not sort forecast values. Sorting emerges as a crucial element of this model's CRPS performance. Without sorting, CRPS increases significantly, by approximately 34%, compared to the full variant.

#### 6.8. AQ-NBEATS ablation studies

Table 5 studies the influence of the following factors in AQ-NBEATS architecture. First, AQ-NBEATS can work with both normalized and non-normalized inputs. Therefore, we explore the case of normalized input, in which case the input time-series is divided by the absolute

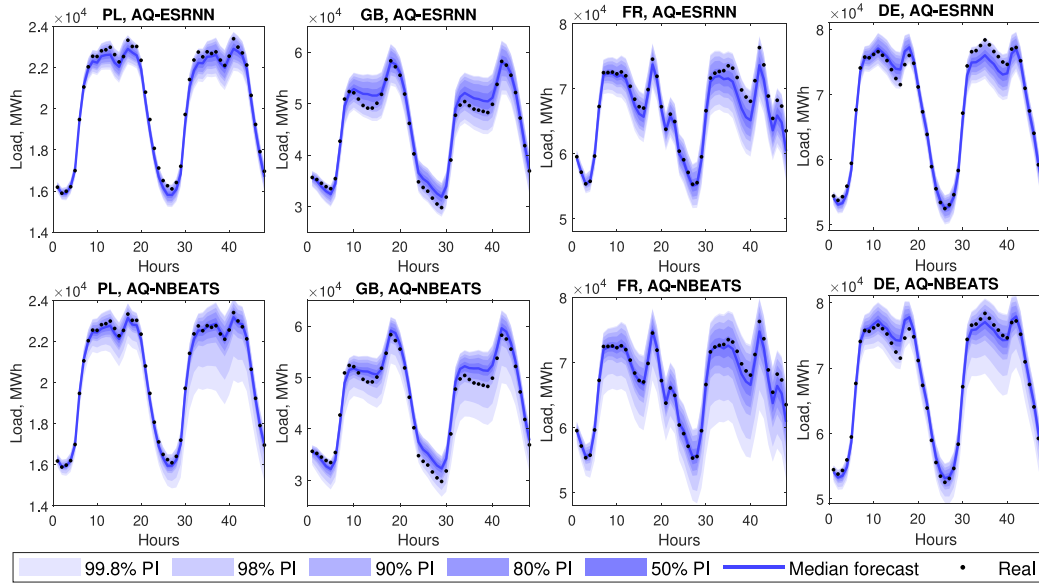


Fig. 10. Examples of quantile forecasts for 30 and 31 January 2018.

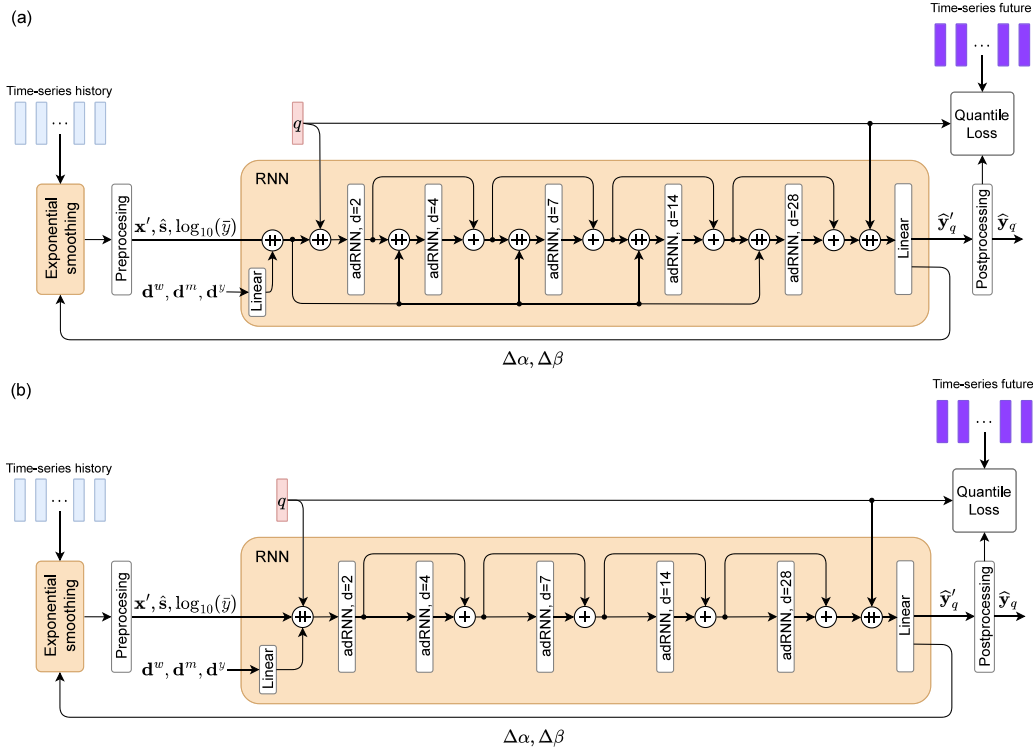


Fig. 11. Reduced versions of AQ-ESRNN: (a) AQ-ESRNN-noQuantInMidLayers, (b) AQ-ESRNN-noInputInMidLayers.

maximum value of the input window (Max-Norm column checkmark) and unnormalized input (Max-Norm column no checkmark). Second, learning supervision can be done either with the MQ loss defined in Eq. (6) (no checkmark in MQN column) or with its normalized MQN variant defined in Eq. (17) (checkmark in MQN column). Finally, we explore the following AQ-NBEATS variants discussed in Section 5.2 and depicted in Fig. 4: (i) AQ-NBEATS-CAT, the quantile level is appended to the input sequence as additional token, (ii) AQ-NBEATS-FiLM, the quantile is inserted in every block through FiLM mechanism, (iii) AQ-NBEATS-OUT, a separate quantile block is appended at the end of the standard N-BEATS.

Looking at Table 5 the following conclusions can be made. First, AQ-NBEATS-FiLM is the most accurate variant, which implies that providing quantile level as input to every block of the architecture is beneficial from the accuracy standpoint. Second, AQ-NBEATS-FiLM and AQ-NBEATS-OUT variants are significantly less sensitive to input normalization (Max-Norm) than the AQ-NBEATS-CAT variant. The main reason for this is that the time-series in the dataset have large magnitude. The quantile probability, distributed in  $[0, 1]$  range, when concatenated with a vector of large magnitude values, has a tendency to be neglected. Based on these observations, we believe that AQ-NBEATS-FiLM and AQ-NBEATS-OUT variants should be preferred in



**Table 4**  
Ablation study of AQ-ESRNN.

|                             | CRPS  | N-CRPS | MAPE |
|-----------------------------|-------|--------|------|
| AQ-ESRNN                    | 195.9 | 1.72   | 2.32 |
| AQ-ESRNN, 4 Layers          | 198.7 | 1.75   | 2.42 |
| AQ-ESRNN, 3 Layers          | 202.7 | 1.77   | 2.41 |
| AQ-ESRNN-noQuantInMidLayers | 203.7 | 1.78   | 2.37 |
| AQ-ESRNN-noInputInMidLayers | 204.2 | 1.79   | 2.41 |
| AQ-ESRNN-dRNN               | 199.2 | 1.74   | 2.36 |
| AQ-ESRNN-LSTM               | 204.0 | 1.79   | 2.41 |
| AQ-ESRNN-unsorted           | 264.2 | 2.34   | 2.35 |

**Table 5**  
Ablation study of the Max-Norm and the concatenation/FiLM based quantile incorporation mechanisms in the N-BEATS architecture.

|                | Max-Norm | MQN | CRPS  | N-CRPS | MAPE |
|----------------|----------|-----|-------|--------|------|
| AQ-NBEATS-CAT  | ✓        |     | 215.3 | 1.94   | 2.62 |
| AQ-NBEATS-CAT  |          |     | 286.7 | 2.58   | 2.57 |
| AQ-NBEATS-FiLM | ✓        |     | 219.3 | 1.98   | 2.68 |
| AQ-NBEATS-FiLM |          |     | 214.1 | 1.86   | 2.51 |
| AQ-NBEATS-OUT  | ✓        |     | 215.7 | 1.94   | 2.61 |
| AQ-NBEATS-OUT  |          |     | 217.2 | 1.9    | 2.56 |
| AQ-NBEATS-CAT  | ✓        | ✓   | 215.5 | 1.86   | 2.52 |
| AQ-NBEATS-CAT  |          | ✓   | 285.1 | 2.46   | 2.48 |
| AQ-NBEATS-FiLM | ✓        | ✓   | 228.1 | 1.94   | 2.62 |
| AQ-NBEATS-FiLM |          | ✓   | 211.2 | 1.84   | 2.47 |
| AQ-NBEATS-OUT  | ✓        | ✓   | 216.4 | 1.86   | 2.52 |
| AQ-NBEATS-OUT  |          | ✓   | 212.5 | 1.84   | 2.48 |

architecture designs due to robustness, accuracy and computational efficiency considerations. Indeed, recall that AQ-NBEATS-CAT deals with the  $B \times T \times Q$  (batch, time, quantiles) input tensor due to the concatenation of quantiles, effectively multiplying the inference compute cost by a factor of  $Q$ . AQ-NBEATS-FiLM has similar compute explosion, but clearly better accuracy. AQ-NBEATS-OUT has comparable accuracy, but compute cost is multiplied by a factor of  $Q$  only for one of the blocks, making it order of magnitude more compute efficient. Finally, the normalized MQ loss defined in (17) may provide improvements, depending on architecture configurations, and can be considered as a hyperparameter to obtain incremental improvements in accuracy.

## 7. Discussion of findings

**Flexibility of AQ-approach to Probabilistic Forecasting.** The review of related work on probabilistic forecasting highlights diverse approaches in Section 1.1. These can be broadly categorized into (i) methods focusing on non-parametric quantile regression with fixed quantile probabilities and (ii) methods that adopt a parametric distribution models allowing them to predict the full distribution and thus arbitrary quantiles. Our methodology falls within the realm of non-parametric models. As such, it can model arbitrary conditional distributions. Yet, it retains the flexibility inherent to parametric techniques by being able to predict an arbitrary quantile. This is achieved through (i) the general parameterization of the forecaster  $f_\theta(\mathbf{x}, q)$ , treating the quantile probability  $q$  as an input parameter and (ii) the learning methodology that trains the model to forecast future trajectories based on input patterns  $\mathbf{x}$  conditioned on randomly selected quantile probabilities  $q$ . Consequently, our AQ-models offer enhanced flexibility and accuracy compared to classical and deep probabilistic forecasting baselines, which is supported by our experimental results.

**Quantile Injection Mechanism.** We experimented with different mechanisms used to inject the quantile probability value in the network. Results for both ESRNN and N-BEATS architectures seem to indicate that the accuracy-optimal injection mechanism exposes every block of the deep neural network to the value of desired quantile probability. The application of quantile only at the input and output works for both ESRNN and N-BEATS, being marginally worse. Additionally,

both simple concatenation and more sophisticated mechanisms such as FiLM layer seem viable for injecting quantile probability. At the same time, concatenation works well in the case when input signal normalization is applied (which is the case of ESRNN by design and it can be used for N-BEATS additionally). FiLM works fine with both normalized and unnormalized data. Finally, injecting quantile probability at the last block of a deep learning architecture seems to be most computationally effective. Given that this is only marginally worse than injecting the quantile at every intermediate architectural block, this method can be preferable in applications requiring fast inference speed. We conjecture that this result implies that most of time-series representation work is independent of the quantile that the network targets. Therefore, the task of time-series representation and the task of quantile generation are largely disjoint and can be handled in a substantially independent way within the forecasting backbone, leading to inference speed gains.

**Quantile Crossing and Sorting.** In general, machine learning models trained for quantile regression may exhibit undesirable quantile crossing behavior, where a forecast for a lower quantile exceeds that of a higher quantile. While the frequency of these occurrences can be reduced by augmenting the loss function with a suitable penalty [65], this does not guarantee that quantile crossing will never occur. Achieving this requires a more complex model, as demonstrated in studies such as [65,66]. Alternatively, post-processing methods can be applied, such as sorting or isotonic projection [67]. Sorting is the simplest method, but it has been shown to have desirable properties, such as ensuring that CRPS never decreases [68]. Interestingly, the two models proposed in this paper, AQ-ESRNN and AQ-NBEATS, respond differently to sorting: it is a crucial step for the former (the metrics improve by 30%, see Table 4) and is practically unimportant for the latter. We leave the in-depth study of root-causes of this phenomenon for future work. In the current work, we conclude that, generally speaking, sorting is a powerful mechanism for improving multi-quantile forecasting, whereas its effectiveness may vary depending on underlying model and problem.

**Probability Sampling Distribution.** Another distinction between the two models lies in the sampling distribution utilized for generating probabilities during training. AQ-ESRNN employs Beta(0.3, 0.3) (please refer to Fig. 3), while AQ-NBEATS remains unaffected by this innovation and utilizes the uniform distribution. Originally, the idea of employing a convex Beta distribution for AQ-ESRNN was introduced to enhance the accuracy of extreme quantiles (where probability is close to 0 or 1), with the expectation that improvements in accuracy in these areas would negatively impact overall accuracy. However, it transpired that this approach also benefits the overall distribution metric, such as CRPS. We speculate that this difference between the two models stems from the fact that AQ-NBEATS employs ReLU nonlinearity, which allows the output to easily grow to any required value, while AQ-ESRNN employs sigmoid and tanh nonlinearities, which may pose more challenges for the output to reach extreme values.

**Differences from Prior Work.** While our method is related to [38, 69] there are also fundamental distinctions. We focus on cross time-series forecasting scenario, presenting the following novel contributions. First, we design advanced any-quantile forecasting architectures for two distinct classes of time-series backbones: ES-RNN and N-BEATS. We do not use an additional Implicit Quantile Network (IQN) to modify the output of a forecasting NN. Instead, we employ a single end-to-end NN that takes the sampled quantile level as one of its inputs. Importantly, our models, particularly those based on recurrent neural networks, explicitly capture temporal dependencies in the quantile space. This is achieved through built-in mechanisms such as hidden states, dilations, attention-based cells, and dynamic exponential smoothing. These features allow the models to learn both short- and long-term temporal patterns relevant for probabilistic forecasting, thereby extending beyond the scope of prior methods. We provide empirical evidence to support the effectiveness of the proposed architectural solutions and study the trade-offs between accuracy and computational complexity of

these approaches. Second, we provide empirical evidence of accuracy improvement of the proposed approaches compared to a large array of benchmarks including statistical models, quantile regression and NGBoost model operating in the local training and inference scenario (looking at one time-series at a time); as well as compared to an array of global approaches based on models from the GluonTS library. Finally, we present theoretical development linking cross-learning over a collection of time-series to the general information fusion problem.

**Additional Data.** While we recognize that meteorological conditions and holiday effects significantly influence electricity demand, our current models do not incorporate these exogenous factors. The primary limitation is data availability: specifically, the lack of archived hourly weather forecasts (up to 48 h ahead) spanning the entire 2006–2018 period across 35 European countries. Although real-time services such as open-meteo.com or yr.no provide current forecasts, to the best of our knowledge, they do not offer historical forecast archives suitable for retrospective modeling. Furthermore, even with access to such data, the inherent uncertainty in weather forecasts — particularly at longer horizons — could introduce additional noise into the learning and inference process. As such, the overall utility of incorporating these inputs remains an open question, which we consider a promising direction for future investigation. Regarding holiday effects, our models currently treat holidays as regular days, effectively acknowledging higher forecast uncertainty during these periods. Prior work has proposed dedicated modeling strategies for holiday corrections [70–72]. We view the integration of such corrections as a practical and modular extension, and leave this for future work.

## 8. Conclusion

We present a general probabilistic forecasting methodology applicable to a wide range of neural network forecasting algorithms. During training, this methodology involves exposing the forecasting algorithm to the task of predicting a randomly sampled quantile. Once trained, the algorithm is capable of providing any arbitrary quantile during inference. We applied this general methodology to two well-known neural forecasting algorithms through novel architectural modifications. Our approach leverages information fusion by employing a cross-learning strategy that combines data from multiple countries. This integration of diverse sources of uncertainty within power systems and complex regional patterns enhances forecasting performance. Empirically, we demonstrated that the proposed modified algorithms significantly outperform several classical and neural forecasting baselines.

Moving forward, our research will focus on extending our current results to additional neural network architectures and other machine learning models, as well as demonstrating the effectiveness of the proposed methodology across different application domains.

## CRedit authorship contribution statement

**Slawek Smyl:** Writing – review & editing, Writing – original draft, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Boris N. Oreshkin:** Writing – review & editing, Writing – original draft, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Paweł Pełka:** Validation, Software, Resources, Investigation. **Grzegorz Dudek:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Formal analysis.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Appendix. Hyperparameters of the baseline models

**Table A.6**

Summary of classical forecasting models used in the study.

| Model type | Parameterization/Notes                                                                                       |
|------------|--------------------------------------------------------------------------------------------------------------|
| ARIMA      | Automatic selection of (p, d, q) and (P, D, Q) via AICc minimization                                         |
| ETS        | Automatic configuration of Error, Trend, and Seasonal components (Additive, Multiplicative, Damped, or None) |
| Theta      | Automatically optimized parameters of the Theta method, Dynamically tuned over time                          |

**Table A.7**

Quantile regression hyperparameters.

| Hyperparameter | Value/Search range |
|----------------|--------------------|
| Solver         | 'highs'            |
| fit_intercept  | True               |
| alpha          | [0.0, 5.0]         |

Hyperparameter alpha was optimized for each country using Optuna with 100 trials.

**Table A.8**

NGBoost hyperparameters (default values with modifications).

| Hyperparameter        | Value                 |
|-----------------------|-----------------------|
| learning_rate         | 0.01 (default: 0.002) |
| n_estimators          | 100 (default: 500)    |
| Base_learner          | DecisionTreeRegressor |
| Minibatch_frac        | 1.0                   |
| Col_sample            | 1.0                   |
| Tol                   | 1e−4                  |
| Validation_fraction   | 0.1                   |
| Early_stopping_rounds | None                  |
| Score                 | LogScore              |
| Natural_gradient      | True                  |

**Table A.9**

Hyperparameters of baseline models from GluonTS (default values).

| Common hyperparameters for all models: |                          |         |
|----------------------------------------|--------------------------|---------|
| Hyperparameter                         | Value                    |         |
| Epochs                                 | 100                      |         |
| Batch size                             | 32                       |         |
| Optimizer                              | Adam                     |         |
| Learning rate                          | 0.001                    |         |
| Dropout                                | 0.1                      |         |
| Model-specific hyperparameters:        |                          |         |
| Model                                  | Hyperparameter           | Value   |
| MLP                                    | Size                     | [40,40] |
|                                        | Activation               | ReLU    |
| DeepAR                                 | Size                     | 40      |
|                                        | Layers                   | 2       |
|                                        | Cell type                | LSTM    |
|                                        | Scaling                  | True    |
| Transformer                            | Model dimension          | 32      |
|                                        | Inner FF dimension scale | 4       |
|                                        | Number of heads          | 8       |
| WaveNet                                | Residual channels        | 24      |
|                                        | Skip channels            | 32      |
|                                        | Number of stacks         | 1       |
|                                        | Epochs                   | 200     |
|                                        | Learning rate            | 0.01    |
| TFT                                    | Size                     | 32      |
|                                        | Variable selection       | True    |
|                                        | Attention heads          | 4       |

## Data availability

The real-world dataset utilized in this study is sourced from the ENTSO-E repository ([www.entsoe.eu/data/power-stats](http://www.entsoe.eu/data/power-stats)). This dataset encompasses hourly electricity demand data for 35 European countries

and we have made this dataset publicly available along with the model code in our GitHub repository <https://github.com/boreshkinai/any-quantile>.

## References

- [1] S.E. Haupt, M. Garcia Casado, M. Davidson, J. Dobschinski, P. Du, M. Lange, T. Miller, C. Mohrlen, A. Motley, R. Pestana, J. Zack, The use of probabilistic forecasts: Applying them in theory and practice, *IEEE Power Energy Mag.* 17 (6) (2019) 46–57, <http://dx.doi.org/10.1109/MPE.2019.2932639>.
- [2] P. Etingov, L. Miller, Z. Hou, Y. Makarov, K. Pennock, P. Beaucage, C. Loutan, A. Motley, Balancing needs assessment using advanced probabilistic forecasts, in: 2018 IEEE International Conference on Probabilistic Methods Applied To Power Systems, PMAPS, 2018, pp. 1–6, <http://dx.doi.org/10.1109/PMAPS.2018.8440392>.
- [3] Q. Wang, A. Tuohy, M. Ortega-Vazquez, M. Bello, E. Ela, D. Kirk-Davidoff, W.B. Hobbs, D.J. Ault, R. Philbrick, Quantifying the value of probabilistic forecasting for power system operation planning, *Appl. Energy* 343 (2023) 121254, <http://dx.doi.org/10.1016/j.apenergy.2023.121254>, URL: <https://www.sciencedirect.com/science/article/pii/S0306261923006189>.
- [4] S. Yamujala, A. Jain, S. Sreekumar, R. Bhakar, J. Mathur, Enhancing power systems operational flexibility with ramp products from flexible resources, *Electr. Power Syst. Res.* 202 (2022) 107599, <http://dx.doi.org/10.1016/j.ejpsr.2021.107599>, URL: <https://www.sciencedirect.com/science/article/pii/S0378779621005800>.
- [5] D. Le, A. Berizzi, C. Bovo, A probabilistic security assessment approach to power systems with integrated wind resources, *Renew. Energy* 85 (2016) 114–123, <http://dx.doi.org/10.1016/j.renene.2015.06.035>, URL: <https://www.sciencedirect.com/science/article/pii/S0960148115300598>.
- [6] M. Beykirch, T. Janke, F. Steinke, Bidding and scheduling in energy markets: Which probabilistic forecast do we need? in: 2022 17th International Conference on Probabilistic Methods Applied To Power Systems, PMAPS, 2022, pp. 1–6, <http://dx.doi.org/10.1109/PMAPS53380.2022.9810632>.
- [7] T. Hong, S. Fan, Probabilistic electric load forecasting: A tutorial review, *Int. J. Forecast.* 32 (3) (2016) 914–938, <http://dx.doi.org/10.1016/j.ijforecast.2015.11.011>, URL: <https://www.sciencedirect.com/science/article/pii/S0169207015001508>.
- [8] H. Khajeh, H. Laaksonen, Applications of probabilistic forecasting in smart grids: A review, *Appl. Sci.* 12 (4) (2022) <http://dx.doi.org/10.3390/app12041823>, URL: <https://www.mdpi.com/2076-3417/12/4/1823>.
- [9] B. Li, J. Zhang, A review on the integration of probabilistic solar forecasting in power systems, *Sol. Energy* 210 (2020) 68–86, <http://dx.doi.org/10.1016/j.solener.2020.07.066>, Special Issue on Grid Integration. URL: <https://www.sciencedirect.com/science/article/pii/S0038092X20307982>.
- [10] R.J. Hyndman, S. Fan, Density forecasting for long-term peak electricity demand, *IEEE Trans. Power Syst.* 25 (2) (2010) 1142–1153, <http://dx.doi.org/10.1109/TPWRS.2009.2036017>.
- [11] J.W. Taylor, R. Buizza, Neural network load forecasting with weather ensemble predictions, *IEEE Trans. Power Syst.* 17 (3) (2002) 626–632, <http://dx.doi.org/10.1109/TPWRS.2002.800906>, URL: <https://www.scopus.com/inward/record.uri?eid=2-s2.0-0036699869&doi=10.1109%2fTPWRS.2002.800906&partnerID=40&md5=019f35fba591b6a47c11ac1e27c6fdb>.
- [12] A. Khoshrou, E.J. Pauwels, Short-term scenario-based probabilistic load forecasting: A data-driven approach, *Appl. Energy* 238 (2019) 1258–1268, <http://dx.doi.org/10.1016/j.apenergy.2019.01.155>, URL: <https://www.sciencedirect.com/science/article/pii/S0306261919301412>.
- [13] J. Xie, T. Hong, Temperature scenario generation for probabilistic load forecasting, *IEEE Trans. Smart Grid* 9 (3) (2018) 1680–1687, <http://dx.doi.org/10.1109/TSG.2016.2597178>.
- [14] J. Xie, T. Hong, T. Laing, C. Kang, On normality assumption in residual simulation for probabilistic load forecasting, *IEEE Trans. Smart Grid* 8 (3) (2017) 1046–1053, <http://dx.doi.org/10.1109/TSG.2015.2447007>.
- [15] C.-L. Hor, S.J. Watson, S. Majithia, Daily load forecasting and maximum demand estimation using ARIMA and GARCH, in: 2006 International Conference on Probabilistic Methods Applied To Power Systems, 2006, pp. 1–6, <http://dx.doi.org/10.1109/PMAPS.2006.360237>.
- [16] Z. Cao, C. Wan, Z. Zhang, F. Li, Y. Song, Hybrid ensemble deep learning for deterministic and probabilistic low-voltage load forecasting, *IEEE Trans. Power Syst.* 35 (3) (2020) 1881–1897, <http://dx.doi.org/10.1109/TPWRS.2019.2946701>.
- [17] B. Liu, J. Nowotarski, T. Hong, R. Weron, Probabilistic load forecasting via quantile regression averaging on sister forecasts, *IEEE Trans. Smart Grid* 8 (2) (2017) 730–737, <http://dx.doi.org/10.1109/TSG.2015.2437877>.
- [18] Y. Yang, S. Li, W. Li, M. Qu, Power load probability density forecasting using Gaussian process quantile regression, *Appl. Energy* 213 (2018) 499–509, <http://dx.doi.org/10.1016/j.apenergy.2017.11.035>, URL: <https://www.sciencedirect.com/science/article/pii/S03062619171316100>.
- [19] Y. He, Y. Qin, S. Wang, X. Wang, C. Wang, Electricity consumption probability density forecasting method based on LASSO-quantile regression neural network, *Appl. Energy* 233–234 (2019) 565–575, <http://dx.doi.org/10.1016/j.apenergy.2018.10.061>, URL: <https://www.sciencedirect.com/science/article/pii/S0306261918316301>.
- [20] R. Liu, T. Chen, G. Sun, S. Mueen, S. Lin, Y. Mi, Short-term probabilistic building load forecasting based on feature integrated artificial intelligent approach, *Electr. Power Syst. Res.* 206 (2022) 107802, <http://dx.doi.org/10.1016/j.ejpsr.2022.107802>, URL: <https://www.sciencedirect.com/science/article/pii/S0378779622000323>.
- [21] Y. He, C. Cao, S. Wang, H. Fu, Nonparametric probabilistic load forecasting based on quantile combination in electrical power systems, *Appl. Energy* 322 (2022) 119507, <http://dx.doi.org/10.1016/j.apenergy.2022.119507>, URL: <https://www.sciencedirect.com/science/article/pii/S0306261922008273>.
- [22] B. Li, Y. Mo, F. Gao, X. Bai, Short-term probabilistic load forecasting method based on uncertainty estimation and deep learning model considering meteorological factors, *Electr. Power Syst. Res.* 225 (2023) 109804, <http://dx.doi.org/10.1016/j.ejpsr.2023.109804>, URL: <https://www.sciencedirect.com/science/article/pii/S0378779623006934>.
- [23] Y. He, R. Liu, H. Li, S. Wang, X. Lu, Short-term power load probability density forecasting method using Kernel-based support vector quantile regression and Copula theory, *Appl. Energy* 185 (2017) 254–266, <http://dx.doi.org/10.1016/j.apenergy.2016.10.079>, URL: <https://www.sciencedirect.com/science/article/pii/S0306261916315239>.
- [24] Y. He, J. Xiao, X. An, C. Cao, J. Xiao, Short-term power load probability density forecasting based on GLRQ-stacking ensemble learning method, *Int. J. Electr. Power Energy Syst.* 142 (2022) 108243, <http://dx.doi.org/10.1016/j.ijepes.2022.108243>, URL: <https://www.sciencedirect.com/science/article/pii/S0142061522002721>.
- [25] G. Dudek, Stacking for probabilistic short-term load forecasting, in: L. Franco, et al. (Eds.), *Computational Science – ICCS 2024*, in: LNCS, vol. 14833, Springer, Cham, 2024, pp. 3–18, [http://dx.doi.org/10.1007/978-3-031-63751-3\\_1](http://dx.doi.org/10.1007/978-3-031-63751-3_1).
- [26] T. Hong, P. Pinson, Y. Wang, R. Weron, D. Yang, H. Zareipour, Energy forecasting: A review and outlook, *IEEE Open Access J. Power Energy* 7 (2020) 376–388, <http://dx.doi.org/10.1109/OAJPE.2020.3029979>.
- [27] Y. Wang, D. Gan, M. Sun, N. Zhang, Z. Lu, C. Kang, Probabilistic individual load forecasting using pinball loss guided LSTM, *Appl. Energy* 235 (2019) 10–20, <http://dx.doi.org/10.1016/j.apenergy.2018.10.078>, URL: <https://www.sciencedirect.com/science/article/pii/S0306261918316465>.
- [28] W. Zhang, H. Quan, D. Srinivasan, An improved quantile regression neural network for probabilistic load forecasting, *IEEE Trans. Smart Grid* 10 (4) (2019) 4425–4434, <http://dx.doi.org/10.1109/TSG.2018.2859749>.
- [29] D. Li, Y. Tan, Y. Zhang, S. Miao, S. He, Probabilistic forecasting method for mid-term hourly load time series based on an improved temporal fusion transformer model, *Int. J. Electr. Power Energy Syst.* 146 (2023) 108743, <http://dx.doi.org/10.1016/j.ijepes.2022.108743>, URL: <https://www.sciencedirect.com/science/article/pii/S0142061522007396>.
- [30] S. Smyl, G. Dudek, P. Pelka, ES-dRNN: A hybrid exponential smoothing and dilated recurrent neural network model for short-term load forecasting, *IEEE Transactions on Neural Networks and Learning Systems* 35 (8) (2024) 11346–11358, <http://dx.doi.org/10.1109/TNNLS.2023.3259149>.
- [31] L. Alfieri, P. De Falco, Wavelet-based decompositions in probabilistic load forecasting, *IEEE Trans. Smart Grid* 11 (2) (2020) 1367–1376, <http://dx.doi.org/10.1109/TSG.2019.2937072>.
- [32] D. van der Meer, M. Shepero, A. Svensson, J. Widén, J. Munkhammar, Probabilistic forecasting of electricity consumption, photovoltaic power generation and net demand of an individual building using Gaussian processes, *Appl. Energy* 213 (2018) 195–207, <http://dx.doi.org/10.1016/j.apenergy.2017.12.104>, URL: <https://www.sciencedirect.com/science/article/pii/S0306261917318275>.
- [33] A. Alexandrov, K. Benidis, M. Bohlke-Schneider, V. Flunkert, J. Gasthaus, T. Januschowski, D.C. Maddix, S. Rangapuram, D. Salinas, J. Schulz, Gluonts: Probabilistic and neural time series modeling in python, *J. Mach. Learn. Res.* 21 (1) (2020) 4629–4634.
- [34] A. Brusaferrri, M. Matteucci, S. Spinelli, A. Vitali, Probabilistic electric load forecasting through Bayesian mixture density networks, *Appl. Energy* 309 (2022) 118341, <http://dx.doi.org/10.1016/j.apenergy.2021.118341>, URL: <https://www.sciencedirect.com/science/article/pii/S0306261921015907>.
- [35] S. Smyl, A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting, *Int. J. Forecast.* 36 (1) (2020) 75–85, <http://dx.doi.org/10.1016/j.ijforecast.2019.03.017>, M4 Competition. URL: <https://www.sciencedirect.com/science/article/pii/S0169207019301153>.
- [36] B.N. Oreshkin, D. Carpo, N. Chapados, Y. Bengio, N-BEATS: Neural basis expansion analysis for interpretable time series forecasting, in: *ICLR*, 2020.
- [37] B.N. Oreshkin, G. Dudek, P. Pelka, E. Turkina, N-BEATS neural network for mid-term electricity load forecasting, *Appl. Energy* 293 (2021) 116918, <http://dx.doi.org/10.1016/j.apenergy.2021.116918>.
- [38] A. Gouttes, K. Rasul, M. Koren, J. Stephan, T. Naghibi, Probabilistic time series forecasting with implicit quantile networks, 2021, [arXiv:2107.03743](https://arxiv.org/abs/2107.03743).

- [39] H. Karimi, J. Nutini, M. Schmidt, Linear convergence of gradient and proximal-gradient methods under the Polyak-Łojasiewicz condition, in: P. Frasconi, N. Landwehr, G. Manco, J. Vreeken (Eds.), *Machine Learning and Knowledge Discovery in Databases*, Springer International Publishing, Cham, 2016, pp. 795–811, [http://dx.doi.org/10.1007/978-3-319-46128-1\\_50](http://dx.doi.org/10.1007/978-3-319-46128-1_50).
- [40] T. Gneiting, R. Ranjan, Comparing density forecasts using threshold-and quantile-weighted scoring rules, *J. Bus. Econom. Statist.* 29 (3) (2011) 411–422, <http://dx.doi.org/10.1198/jbes.2010.08110>.
- [41] S. Smyl, G. Dudek, P. Pelka, ES-dRNN with dynamic attention for short-term load forecasting, in: 2022 International Joint Conference on Neural Networks, IJCNN, 2022, pp. 1–8, <http://dx.doi.org/10.1109/IJCNN55064.2022.9889791>.
- [42] A.-A. Semenov, E. Spiliotis, S. Makridakis, V. Assimakopoulos, Investigating the accuracy of cross-learning time series forecasting methods, *Int. J. Forecast.* 37 (3) (2021) 1072–1084, <http://dx.doi.org/10.1016/j.ijforecast.2020.11.009>, URL: <https://www.sciencedirect.com/science/article/pii/S0169207020301850>.
- [43] H. Hewamalage, C. Bergmeir, K. Bandara, Global models for time series forecasting: A simulation study, *Pattern Recognit.* 124 (2022) 108441, <http://dx.doi.org/10.1016/j.patcog.2021.108441>, URL: <https://www.sciencedirect.com/science/article/pii/S0031320321006178>.
- [44] R. Hyndman, G. Athanasopoulos, *Forecasting: Principles and Practice*, second ed., OTexts, Australia, 2018.
- [45] E. Perez, F. Strub, H. de Vries, V. Dumoulin, A.C. Courville, Film: Visual reasoning with a general conditioning layer, in: AAAI, 2018, <http://dx.doi.org/10.1609/aaai.v32i1.11671>.
- [46] T. Poplawski, G. Dudek, J. Lyp, Forecasting methods for balancing energy market in Poland, *Int. J. Electr. Power Energy Syst.* 65 (2015) 94–101, <http://dx.doi.org/10.1016/j.ijepes.2014.09.029>.
- [47] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimesheine, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, Pytorch: An imperative style, high-performance deep learning library, in: *NeurIPS*, 2019, pp. 8024–8035.
- [48] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2015, [arXiv: 1412.6980](https://arxiv.org/abs/1412.6980).
- [49] W. Falcon, The PyTorch Lightning team, PyTorch Lightning, 2019, <http://dx.doi.org/10.5281/zenodo.3828935>, URL: <https://github.com/Lightning-AI/lightning>.
- [50] G. Dudek, Pattern similarity-based methods for short-term load forecasting – Part 2: Models, *Appl. Soft Comput.* 36 (2015) 422–441, <http://dx.doi.org/10.1016/j.asoc.2015.07.035>.
- [51] G. Dudek, Short-term load forecasting using theta method, in: 14th Int. Conf. on Forecasting in Power Engineering 2018, E3S Web Conf., Vol. 84, 2019, <http://dx.doi.org/10.1051/e3sconf/20198401004>.
- [52] R. Koenker, K.F. Hallock, Quantile regression, *J. Econ. Perspect.* 15 (4) (2001) 143–156, <http://dx.doi.org/10.1257/jep.15.4.143>.
- [53] T. Duan, A. Avati, D.Y. Ding, K.K. Thai, S. Basu, A.Y. Ng, A. Schuler, NGBoost: Natural gradient boosting for probabilistic prediction, in: H.D. III, A. Singh (Eds.), *Proceedings of the 37th International Conference on Machine Learning, ICML*, in: *Proceedings of Machine Learning Research*, Vol. 119, PMLR, 2020, pp. 2690–2700, URL: <https://proceedings.mlr.press/v119/duan20a.html>.
- [54] G. Dudek, Pattern-based local linear regression models for short-term load forecasting, *Electr. Power Syst. Res.* 130 (2016) 139–147, <http://dx.doi.org/10.1016/j.epsr.2015.09.001>.
- [55] D. Salinas, V. Flunkert, J. Gasthaus, T. Januschowski, DeepAR: Probabilistic forecasting with autoregressive recurrent networks, *Int. J. Forecast.* 36 (3) (2020) 1181–1191, <http://dx.doi.org/10.1016/j.ijforecast.2019.07.001>.
- [56] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, et al., Attention is all you need, in: *Proceedings of 31st Conference on Neural Information Processing Systems, NIPS 2017*, 2017, pp. 5998–6008.
- [57] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, K. Kavukcuoglu, WaveNet: A generative model for raw audio, 2016, [arXiv:1609.03499](https://arxiv.org/abs/1609.03499).
- [58] B. Lim, S. Arik, N. Loeff, T. Pfister, Temporal fusion transformers for interpretable multi-horizon time series forecasting, *Int. J. Forecast.* 37 (4) (2021) 1748–1764, <http://dx.doi.org/10.1016/j.ijforecast.2021.03.012>.
- [59] R. Hyndman, G. Athanasopoulos, *Forecasting: Principles and Practice*, third ed., OTexts, Melbourne, Australia, 2021, (Accessed 28 December 2023).
- [60] J.A. Fiorucci, T.R. Pellegrini, F. Louzada, F. Petropoulos, A.B. Koehler, Models for optimising the theta method and their relationship to state space models, *Int. J. Forecast.* 32 (4) (2016) 1151–1161, <http://dx.doi.org/10.1016/j.ijforecast.2016.02.005>.
- [61] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, Édouard Duchesnay, Scikit-learn: Machine learning in Python, *J. Mach. Learn. Res.* 12 (85) (2011) 2825–2830, URL: <http://jmlr.org/papers/v12/pedregosa11a.html>.
- [62] F.X. Diebold, R.S. Mariano, Comparing predictive accuracy, *J. Bus. Econom. Statist.* 13 (3) (1995) 253–263, <http://dx.doi.org/10.1080/07350015.1995.10524599>.
- [63] S. Makridakis, E. Spiliotis, V. Assimakopoulos, Z. Chen, A. Gaba, I. Tsetlin, R.L. Winkler, The M5 uncertainty competition: Results, findings and conclusions, *Int. J. Forecast.* 38 (4) (2022) 1365–1385, <http://dx.doi.org/10.1016/j.ijforecast.2021.10.009>, Special Issue: M5 competition. URL: <https://www.sciencedirect.com/science/article/pii/S0169207021001722>.
- [64] T. Gneiting, A.E. Raftery, Strictly proper scoring rules, prediction, and estimation, *J. Amer. Statist. Assoc.* 102 (477) (2007) 359–378, <http://dx.doi.org/10.1198/016214506000001437>.
- [65] W. Tang, G. Shen, Y. Lin, J. Huang, Nonparametric quantile regression: Non-crossing constraints and conformal prediction, 2022, [arXiv:2210.10161](https://arxiv.org/abs/2210.10161).
- [66] Y. Park, D. Maddix, F.-X. Aubet, K. Kan, J. Gasthaus, Y. Wang, Learning quantile functions without quantile crossing for distribution-free time series forecasting, 2022, [arXiv:2111.06581](https://arxiv.org/abs/2111.06581).
- [67] R. Fakoor, T. Kim, J. Mueller, A.J. Smola, R.J. Tibshirani, Flexible model aggregation for quantile regression, 2023, [arXiv:2103.00083](https://arxiv.org/abs/2103.00083).
- [68] V. Chernozhukov, I. Fernández-Val, A. Galichon, Quantile and probability curves without crossing, *Econom.* 78 (3) (2010) 1093–1125, <http://dx.doi.org/10.3982/ecta7880>.
- [69] N. Tagasovska, D. Lopez-Paz, Single-model uncertainties for deep learning, in: *Proc NeurIPS*, 2019.
- [70] J. Son, J. Cha, H. Kim, Y.-M. Wi, Day-ahead short-term load forecasting for holidays based on modification of similar days' load profiles, *IEEE Access* 10 (2022) 17864–17880.
- [71] J. Gunawan, C.-Y. Huang, An extensible framework for short-term holiday load forecasting combining dynamic time warping and LSTM network, *IEEE Access* 9 (2021).
- [72] S. Arora, J. Taylor, Short-term load forecasting for special days using rule-based models: A case study for France, *European J. Oper. Res.* (2018).